UNIVERSIDAD PRIVADA DE TACNA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS

INFORME TÉCNICO – II UNIDAD

DISEÑO DE LA SOLUCIÓN TECNOLÓGICA: ARQUITECTURA E INFRAESTRUCTURA DE n8n SOBRE LINUX PARA LA AUTOMATIZACIÓN DE PEDIDOS POR WHATSAPP EN RETEQUE-OS

CURSO:
Sistemas Operativos II

INTEGRANTES:

- Milton H Flores Chino
- Kevin Elvis Hinojosa CalloApaza
- Manuel Andree Dongo Palza
- Usher Damiron Halanocca Rojas
- Yerry Maycol Tarqui Collatupa

DOCENTE:
Dr. Taco Coayla Renzo Alberto

TACNA - PERÚ
2026

ÍNDICE

1. RESUMEN EJECUTIVO....4
2. PLANTEAMIENTO DEL PROBLEMA....4
2.1 Situación actual....4
2.2 Necesidad identificada....5
2.3 Justificación de la propuesta....5
2.4 Alcance, supuestos y limitaciones....6
3. OBJETIVOS....6
3.1 Objetivo general....6
3.2 Objetivos específicos....6
4. REQUERIMIENTOS....7
4.1 Requerimientos funcionales....7
4.2 Requerimientos no funcionales....8
5. DISEÑO DE LA ARQUITECTURA....9
5.1 Trazabilidad entre la investigación y el diseño....9
5.2 Arquitectura lógica....12
5.3 Arquitectura general y arquitectura física....13
5.4 Componentes principales....14
5.5 Decisiones de arquitectura y alternativas descartadas....14
5.6 Flujo de información....16
6. DISEÑO DE INFRAESTRUCTURA....18
6.1 Servidores....18
6.2 Equipos....19
6.3 Sistemas operativos....19
6.4 Almacenamiento....19
6.5 Virtualización....20
6.6 Servicios requeridos....21
7. DISEÑO DE RED....21
7.1 Topología....21
7.2 Segmentación....22
7.3 Direccionamiento IP....23
7.4 VLAN....23
7.5 Firewall....24
7.6 Acceso remoto....25
8. DISEÑO DE SEGURIDAD....25
8.1 Riesgos identificados....25
8.2 Amenazas y controles propuestos....26
8.3 Gestión de accesos....29
8.4 Respaldo y recuperación....29
8.5 Monitoreo y auditoría....31
9. DISEÑO OPERATIVO....31
9.1 Roles y responsabilidades....31
9.2 Procedimientos de operación....32
9.3 Procedimientos de mantenimiento....33
9.4 Gestión de incidentes....33
10. PLAN DE IMPLEMENTACIÓN....34
10.1 Actividades y dependencias....34
10.2 Recursos necesarios....35
10.3 Cronograma....36
11. PLAN DE PRUEBAS....36
11.1 Estrategia y entorno de pruebas....36
11.2 Casos de prueba....37
11.3 Criterios de aceptación globales....39
11.4 Verificación analítica previa del diseño....39
11.5 Prototipo de verificación del diseño (prueba de concepto)....40
12. ANÁLISIS DE FACTIBILIDAD....41
12.1 Factibilidad técnica....41
12.2 Factibilidad económica....41

12.3 Factibilidad operativa.....42
13. CONCLUSIONES.....43
14. REFERENCIAS BIBLIOGRÁFICAS.....43
ANEXO A. CONFIGURACIONES DE REFERENCIA.....51
ANEXO B. EVIDENCIA DEL PROTOTIPO.....53

1. RESUMEN EJECUTIVO

RETEQUE-OS es el ecosistema comercial digital de “Retequeños” (Tacna): una tienda web y aplicación web progresiva (PWA) para clientes, un panel administrativo (pantalla de cocina o KDS, caja y catálogo) y una interfaz de programación (API) de negocio en Node.js que concentra las reglas y los datos. El problema que aborda este informe es que la atención de pedidos por WhatsApp (consultas, cotización, cierre, recepción de comprobantes y avisos de estado) todavía depende de la intervención manual del personal, lo que limita la rapidez de respuesta, la trazabilidad y la capacidad de crecer sin aumentar la carga administrativa.

La solución propuesta es un **diseño de infraestructura y arquitectura para automatizar ese proceso con n8n Community autoalojado en un servidor privado virtual (VPS) Linux**, bajo el principio “**n8n coordina, la API decide**”. El canal es la WhatsApp Cloud API oficial de Meta; los pedidos y eventos se guardan en la misma transacción (patrón *outbox*) en PostgreSQL de Supabase; un *outbox worker* entrega los eventos a n8n, que ejecuta los flujos de mensajería y vuelve a consultar a la API mediante endpoints internos. Un comprobante recibido nunca equivale a un pago confirmado: la verificación final es humana o proviene de una conciliación fiable.

Desde la perspectiva de Sistemas Operativos II, el servidor es un Ubuntu Server 24.04 LTS (2 vCPU, 4 GB de RAM, 80 GB SSD) donde cada servicio corre en un contenedor Docker aislado mediante *namespaces* y *cgroups*, detrás de Caddy (cifrado TLS en los puertos 80 y 443), con cortafuegos de dos niveles, acceso administrativo por VPN WireGuard y SSH con claves, respaldos cifrados diarios hacia Cloudflare R2 y metas de recuperación de **RPO ≤ 24 h** (pérdida máxima de datos) y **RTO ≤ 2 h** (tiempo máximo de restablecimiento). El costo recurrente estimado es de **S/ 264,19 al mes (S/ 3 170,24 al año)** más los mensajes de WhatsApp, que se modelan por escenarios (≈ S/ 104 con 500 pedidos mensuales y ≈ S/ 728 con 2 000), y la licencia de software es S/ 0.

El diseño se sustenta en las 40 fuentes de la I Unidad y en diez referencias normativas y técnicas, e incluye requerimientos con trazabilidad hacia las pruebas, arquitectura lógica y física con las alternativas descartadas, infraestructura, red (con direccionamiento IP verificado, segmentación y VLAN), seguridad alineada con ISO/IEC 27001:2022 e ISO/IEC 27005:2022, operación, plan de implementación, plan de pruebas y análisis de factibilidad. Todo lo aquí descrito es un **diseño** cuyas metas se medirán en la III Unidad en el entorno real. La verificación realizada hasta ahora es analítica (sección 11.4) y de un prototipo mínimo (sección 11.5), cuyas cifras de rendimiento provienen de un entorno de desarrollo y no de la VPS.

Palabras clave: n8n; WhatsApp Cloud API; VPS Linux; Docker; aislamiento por contenedor; respaldo y recuperación; RETEQUE-OS.

Materiales del proyecto. El repositorio público del grupo está en https://github.com/Yerrytc/N8N_SO2 (artículo de la I Unidad en Word y PDF, diapositivas y enlace a la carpeta de fuentes), y las 40 fuentes de la I Unidad están en la carpeta de Google Drive https://drive.google.com/drive/folders/1D0T0N4jpK7AIfQZyX4QkX4prb8k3Pmf_?usp=sharing. El informe de la II Unidad, sus diapositivas, las figuras y el prototipo (código y evidencia de la sección 11.5) se publican en el mismo repositorio.

2. PLANTEAMIENTO DEL PROBLEMA

2.1 Situación actual

RETEQUE-OS cuenta con una tienda web/PWA para catálogo, carrito y pedidos, un panel administrativo para cocina (KDS), caja y catálogo, y una API de negocio con las reglas de precios, stock y estados. La automatización con n8n y los avisos por WhatsApp figuran como componentes “por implementar”. En el documento de diseño del proyecto (Equipo RETEQUE-OS, 2026) la venta asistida por chat exige que el administrador atienda cada consulta frecuente, calcule importes, solicite el pago, reciba el comprobante y lo asocie al

pedido correcto; todas esas tareas hoy dependen de una persona y de la memoria de la conversación.

El estudio documental de la I Unidad (Flores Chino et al., 2026) mostró que n8n es una plataforma de orquestación consolidada, pero también que el autoalojamiento traslada al administrador las tareas de instalación, exposición de servicios, credenciales, almacenamiento, actualizaciones, TLS, copias de seguridad, permisos, registros y recuperación, y que la elección de la infraestructura y de los mecanismos de verificación de pagos quedó para una fase posterior de diseño. Ese conjunto de responsabilidades es, precisamente, un problema de sistemas operativos.

Línea base de referencia. Las fuentes de la I Unidad muestran el orden de magnitud del problema en otros negocios. Antes de automatizar, el tiempo de respuesta al cliente era de 8,24 horas en una constructora inmobiliaria (Muñoz Ingaruca, 2026), la espera en un proceso de ventas era de 688,84 s (Nuñez Cartolin, 2021) y emitir una factura por WhatsApp tomaba de 9 a 10 minutos (Alvarado Huamán et al., 2025); después, esos valores bajaron a 20,41 s, 11,46 s y menos de 2 minutos, respectivamente. Son contextos distintos y no se trasladan como meta: sirven para dimensionar el problema. Para RETEQUE-OS no existen mediciones propias; el tiempo medio de respuesta manual y el número real de pedidos por día se medirán al inicio de la III Unidad (actividad A01) y serán la línea base de la prueba PT-18.

2.2 Necesidad identificada

Se necesita un entorno de ejecución que permita que la automatización de pedidos funcione de forma continua, segura y recuperable con un costo que un negocio pequeño pueda sostener. Del análisis del caso y de la literatura se derivan cuatro necesidades concretas:

- **Continuidad:** el canal de WhatsApp no debe interrumpirse por un reinicio o por la caída de un proceso; el webhook debe responder rápido y sin perder eventos.
- **Aislamiento y control de recursos:** varios servicios (API, worker, n8n, base de datos y proxy) comparten una sola máquina; un fallo o exceso de consumo de uno no debe afectar al resto (Bui, 2015; Chen et al., 2026).
- **Seguridad operativa:** exposición mínima de puertos, TLS, autenticación de webhooks, protección de credenciales y aprobación humana de pagos (Tang et al., 2026; Fendley et al., 2026).
- **Recuperación probada:** copias externas y restauración ensayada, de modo que una pérdida del servidor no implique la pérdida del negocio.

2.3 Justificación de la propuesta

Técnica. La literatura revisada documenta despliegues muy similares: Muñoz Ingaruca (2026) diseñó un Droplet de 2 vCPU, 4 GB de RAM y 80 GB de disco para un bot con n8n, WhatsApp Business API y base de datos, e implementó Docker, Compose, n8n y Caddy en una VPS Linux; Ruhallah et al. (2025) y Chen et al. (2026) describen arquitecturas con webhook, n8n y contenedores sobre servidores propios. Estas fuentes respaldan que la combinación n8n + Docker + proxy inverso en una sola VPS es viable para cargas pequeñas.

Económica. Las herramientas base no tienen costo de licencia (n8n Community, Docker, Caddy, PostgreSQL, Ubuntu) y los servicios en la nube tienen un costo mensual predecible. La única partida variable es la mensajería de WhatsApp, que se modela por escenarios.

Académica. El diseño permite aplicar los contenidos del curso: procesos y servicios (systemd), aislamiento (namespaces y cgroups), memoria y almacenamiento, redes y cortafuegos, seguridad de usuarios y archivos, respaldos y monitoreo. Además, la II Unidad exige que lo diseñado sea implementado y validado en la III Unidad, por lo que cada decisión se formula de manera verificable.

Brecha identificada. Las 40 fuentes de la I Unidad documentan n8n con WhatsApp, Telegram y otros canales, y algunas describen el servidor (Muñoz Ingaruca, 2026; Chen et al., 2026; Patiño et al., 2026), pero en la revisión del equipo no se identificó un diseño que reúna, en un solo documento, el aislamiento de los servicios, la segmentación de red, la política de respaldo con metas de recuperación, la operación y el plan de pruebas de un n8n autoalojado que atienda pedidos por WhatsApp. Además, solo 167 de 6 003 flujos públicos analizados incluyen aprobación humana explícita (Tang et al., 2026), lo que justifica diseñar esa verificación desde la arquitectura. Este informe cubre esa brecha a nivel de diseño.

2.4 Alcance, supuestos y limitaciones

- **Alcance (MVP):** un restaurante, un número de WhatsApp, catálogo precargado, pago por transferencia o billetera con comprobante, validación humana y panel para 1 a 5 operadores.
- **Fuera del alcance inicial:** confirmación automática de pagos por OCR, integración con POS o facturación electrónica, mensajes masivos y agentes de IA que modifiquen datos. La IA es opcional y no forma parte del MVP; si se incorpora, solo interpretará texto y devolverá datos estructurados que el backend validará.
- **Supuestos:** tráfico entre 100 y 2 000 pedidos mensuales y cuatro mensajes de servicio por pedido; disponibilidad de una cuenta de WhatsApp Business verificada; dominio propio.
- **Limitaciones:** es un diseño sin mediciones propias; los precios y versiones corresponden a la fecha de elaboración y deben confirmarse al contratar; la evidencia de la literatura proviene de contextos distintos y no se promedia.

3. OBJETIVOS

3.1 Objetivo general

Diseñar, hasta la entrega de la II Unidad (noviembre de 2026), la arquitectura, la infraestructura, la red, la seguridad y la operación de una solución basada en n8n autoalojado sobre un servidor Linux que automatice la atención de pedidos por WhatsApp de RETEQUE-OS, con metas medibles de disponibilidad (≥ 99,0 %), respuesta del webhook (< 1 s) y recuperación (RPO ≤ 24 h y RTO ≤ 2 h), sustentada en las 40 fuentes de la I Unidad y verificable con 24 pruebas en la III Unidad.

3.2 Objetivos específicos

1. Especificar 19 requerimientos funcionales y 12 no funcionales, cada uno con prioridad, métrica y prueba asociada (sección 4).
2. Diseñar la arquitectura lógica y física con los cuatro diagramas exigidos (arquitectura general, topología de red, seguridad y despliegue) y justificar cada decisión frente a alternativas descartadas (sección 5).
3. Dimensionar la infraestructura de modo que la suma de los límites de memoria de los contenedores (2 784 MB) quede por debajo de los 4 GB del servidor y el disco conserve al menos 50 % libre (sección 6).
4. Diseñar la red con nueve segmentos sin solapamientos de direcciones y con una exposición a Internet limitada a tres puertos: 80/tcp, 443/tcp y 51820/udp (sección 7).
5. Valorar 12 riesgos con controles mapeados a ISO/IEC 27001:2022 y definir el respaldo y la recuperación con RPO ≤ 24 h y RTO ≤ 2 h (sección 8).

6. Establecer el diseño operativo, un plan de implementación de ocho semanas con 19 actividades, 24 casos de prueba con criterios de aceptación y un análisis de factibilidad con costos en soles (secciones 9 a 12).

4. REQUERIMIENTOS

Los requerimientos se obtuvieron del documento de arquitectura del proyecto (Equipo RETEQUE-OS, 2026) y de la revisión de la I Unidad. La prioridad sigue el criterio MoSCoW: M (debe), S (debería) y C (podría).

4.1 Requerimientos funcionales

La Tabla 1 enumera los 19 requerimientos funcionales con su prioridad, el componente responsable y la prueba que los verifica.

Tabla 1
Requerimientos funcionales de la solución

ID	Requerimiento	Prior.	Componente · prueba
RF-01	Recibir mensajes y archivos de WhatsApp en un webhook que verifica el token (GET) y la firma HMAC X-Hub-Signature-256 (POST).	M	API de negocio · PT-11
RF-02	Identificar o crear al cliente y la conversación por teléfono; descartar mensajes repetidos por identificador (wamid).	M	API, Supabase · PT-12
RF-03	Consultar el catálogo vigente: productos, variantes, precios y disponibilidad.	M	API, Supabase · PT-18
RF-04	Calcular la cotización y los totales en el backend (precios en enteros) y solicitar confirmación del cliente.	M	API · PT-18
RF-05	Registrar el pedido y su evento en la misma transacción (outbox).	M	API, Supabase · PT-23
RF-06	Entregar los eventos pendientes a n8n con reintentos y espera exponencial, sin efectos duplicados.	M	Outbox worker · PT-23
RF-07	Orquestar los flujos de mensajería (enrutamiento, carrito, cierre, pago, resultado, derivación humana y errores).	M	n8n · PT-18
RF-08	Enviar mensajes de salida por la WhatsApp Cloud API y registrar su estado de entrega.	M	API, Meta · PT-18
RF-09	Recibir el comprobante, almacenarlo, asociarlo al pedido, pasar a UNDER_REVIEW y generar una alerta.	M	API, Storage · PT-15
RF-10	Permitir la verificación humana del pago (aprobar o rechazar) con registro de quién y cuándo.	M	Panel, API · PT-15
RF-11	Mostrar los pedidos en el panel y el KDS y permitir cambios de estado con control de versión.	M	Panel, API · PT-18
RF-12	Derivar la conversación a una persona cuando el flujo no pueda resolverla.	S	n8n, Panel · PT-18
RF-13	Notificar al cliente los cambios de estado (confirmado, en preparación, listo o en camino).	M	n8n, API · PT-18

ID	Requerimiento	Prior.	Componente · prueba
RF-14	Enviar una encuesta breve de satisfacción (1 a 5) después de la entrega.	S	n8n · PT-22
RF-15	Respaldar automáticamente la base de n8n, sus flujos, la configuración y los archivos, y documentar la restauración.	M	Host, R2 · PT-19
RF-16	Monitorear la salud de los servicios, el uso de CPU, RAM y disco, y los fallos de envío, con alertas.	M	Host, proveedor · PT-24
RF-17	Gestionar usuarios y roles (ADMIN, CAJA, OPERADOR, COCINA sin acceso a comprobantes, AUDITOR).	M	API, Panel · PT-15
RF-18	Registrar una bitácora de auditoría de cambios de estado, accesos y acciones administrativas.	M	API, host · PT-15
RF-19	Interpretar texto libre con un modelo de lenguaje y devolver JSON validable (opcional; no forma parte del MVP).	C	n8n, API · fuera del MVP

Nota. Elaboración propia a partir de Equipo RETEQUE-OS (2026). M = debe; S = debería; C = podría.

4.2 Requerimientos no funcionales

Cada requerimiento no funcional tiene una meta, un método de verificación y una referencia. Las metas de disponibilidad, tiempo y satisfacción son las fijadas en el diseño del equipo; las demás son propuestas de este informe, y todas se comprobarán con el plan de pruebas (sección 11) y las categorías de calidad de ISO/IEC 25010 (ISO/IEC, 2011).

La Tabla 2 fija las metas medibles de los atributos de calidad y la prueba que verifica cada una.

Tabla 2
Requerimientos no funcionales

Atributo	Requerimiento y meta	Verificación	Referencia
Disponibilidad	RNF-01 Servicio 24/7 con reinicio automático y healthchecks. Meta propuesta: ≥ 99,0 % mensual (≈ 7,2 h de indisponibilidad tolerada). RNF-02 RPO ≤ 24 h y RTO ≤ 2 h.	Corte controlado de contenedores; restauración cronometrada (PT-06, PT-07, PT-19, PT-20).	Muñoz Ingaruca (2026)
Seguridad	RNF-03 Solo 80/443 y 51820/udp visibles desde Internet; TLS válido; SSH únicamente por VPN y con claves. RNF-04 Webhook con firma verificada; secretos fuera del repositorio; aislamiento por contenedor sin privilegios; comprobante ≠ pago.	Aislamiento, escaneo de puertos, prueba de firma inválida y revisión de configuración (PT-03 a PT-05 y PT-08 a PT-15).	Bui (2015); Tang et al. (2026)

Atributo	Requerimiento y meta	Verificación	Referencia
Rendimiento	RNF-05 Webhook de Meta responde HTTP 200 en < 1 s. RNF-06 Confirmación del pedido ≤ 2 min; mensajes enviados correctamente ≥ 95 %; pedidos atendidos a tiempo ≥ 90 %. Un pedido está a tiempo si queda listo en 15 a 20 min desde su confirmación y, en delivery o recojo, se entrega en el plazo prometido según zona y horario.	Mediciones de latencia y prueba de carga con 30 conversaciones simultáneas (PT-16 a PT-18).	Ruhallah et al. (2025)
Escalabilidad	RNF-07 Soportar 2 000 pedidos/mes (≈ 8 000 mensajes de servicio) en la VPS inicial. RNF-08 Escalar a 4 vCPU/8 GB o a n8n en modo cola (Redis y workers) si la CPU > 70 % o la RAM > 80 % de forma sostenida.	Prueba de carga y revisión de métricas de consumo (PT-17).	Venkiteela (2025); Equipo RETEQUE-OS (2026)
Mantenibilidad	RNF-09 Versiones de imágenes fijadas y configuración versionada en Git; actualizaciones con respaldo previo y reversión documentada. RNF-10 Procedimientos y diagramas actualizados.	Revisión de Compose y repositorio; ensayo de actualización y reversión (PT-21).	Casaclang et al. (2026)
Usabilidad	RNF-11 Flujo conversacional de pedido en pocos pasos y panel de caja/KDS legible. RNF-12 Satisfacción del cliente (CSAT) ≥ 4/5 y escala de usabilidad del sistema (SUS) ≥ 68 en la prueba con operadores.	Encuesta CSAT y cuestionario SUS (PT-22).	Andaluz Freire y Ulloa Gavilanes (2026); Jondec Delgado et al. (2026)

Nota. Elaboración propia. Los umbrales de 99,0 %, 70 %, 80 % y SUS ≥ 68 son propuestas de diseño que deberán validarse.

5. DISEÑO DE LA ARQUITECTURA

5.1 Trazabilidad entre la investigación y el diseño

La Tabla 3 muestra qué hallazgo de las 40 fuentes de la I Unidad sustenta cada decisión de arquitectura y qué se decidió en consecuencia. Cuando la fuente describe otro dominio u otra escala, se usa como referencia de práctica y no como prueba del desempeño de este diseño; las prepublicaciones y los resultados autoinformados se señalan como tales.

Tabla 3
Trazabilidad entre la evidencia de la I Unidad y las decisiones de diseño

Decisión de diseño	Evidencia en la literatura	Aplicación en este diseño
n8n Community autoalojado como orquestador	Moreno (2025) compara n8n con Zapier y Make y destaca su edición Community autoalojable; Venkiteela (2025) lo contrasta con Node-RED, Airflow y Boomi; Flores Chino et al. (2026) concluyen que el autoalojamiento da control pero exige gestionar credenciales, TLS, copias y recuperación.	Sin licencia por uso interno; las tareas operativas se asumen en las secciones 8 y 9.
Una VPS con Docker Compose y proxy inverso	Muñoz Ingaruca (2026) diseñó un Droplet de 2 vCPU, 4 GB y 80 GB para un bot con n8n y WhatsApp e implementó Docker, Compose, n8n y Caddy en Ubuntu 24.04 con SSH por clave pública; Ruhallah et al. (2025) también despliegan su chatbot en una VPS.	Mismo dimensionamiento y patrón: Caddy en 80/443 y n8n interno.
Aislamiento por contenedor	Bui (2015) explica que Docker aísla con cinco *namespaces* (montaje, nombre de host, IPC, PID y red), limita recursos con cgroups y advierte sobre los contenedores privilegiados.	Sin privilegios, usuario no root, capacidades mínimas y límites de CPU y memoria.
Redes Docker segmentadas	Chen et al. (2026) despliegan n8n, FastAPI, Ollama y PostgreSQL con Docker Compose en una red puente aislada.	Tres redes: net_edge, net_app y net_data (interna).
Configuración y secretos	Casaclang et al. (2026) autoalojan n8n con Docker Compose y un archivo .env; su ejemplo ejecuta el contenedor como root, práctica que este diseño evita.	.env con permisos 600 fuera de Git; llave de cifrado de n8n respaldada.
HTTPS, token y mínimo privilegio	Patiño et al. (2026) ejecutan n8n en Docker, consumen una API con token sobre HTTPS y leen los datos de origen mediante vistas de solo lectura.	API con tokens y TLS; rol de solo lectura para los respaldos.
Crecimiento por etapas	Venkiteela (2025) distingue contenedor único, servidor y modo cola con Redis y workers, y reporta más CPU y RAM al subir de 10 a 100 trabajos simultáneos.	Contenedor único en el MVP; modo cola al superar los umbrales de RNF-08.
Webhook → n8n → mensajería	Ruhallah et al. (2025) usan WhatsApp, webhook y n8n con limitaciones en picos de consultas; Alvarado Huamán et al. (2025) combinan WhatsApp, n8n y la API de la SUNAT; Ramadhani et al. (2025) envían resúmenes por la API de WhatsApp desde hojas de cálculo.	Webhook en la API (respuesta rápida) y n8n desacoplado por outbox.

Decisión de diseño	Evidencia en la literatura	Aplicación en este diseño
Base relacional y panel de pedidos	Fitriani y Suyatno (2026) usan WhatsApp, n8n, PostgreSQL y un tablero; Sisephaputra et al. (2026) incluyen aprobaciones humanas; Andaluz Freire y Ulloa Gavilanes (2026) estudian pedidos con n8n, IA y WhatsApp Business.	Supabase PostgreSQL, panel con roles y verificación humana.
Aprobación humana y control de la IA	Tang et al. (2026) hallaron aprobación humana explícita en solo 167 de 6 003 flujos públicos; Fendley et al. (2026) y de Freitas y Lotufo (2024) muestran riesgos de inyección en agentes.	Comprobante ≠ pago; IA fuera del MVP y sin autoridad sobre precios ni pagos.
El estado del pedido no lo decide el modelo de lenguaje	Hudeček y Dušek (2023) hallaron que los modelos de lenguaje rinden menos que los modelos especializados al seguir el estado de la conversación; Yao et al. (2023) y Schick et al. (2023) proponen que el modelo invoque herramientas externas; Widiyanti et al. (2026) separan la interpretación del modelo de la ejecución de herramientas predefinidas.	La API mantiene la máquina de estados; la IA, si se usa, solo interpreta texto.
Límite de conocimiento y derivación humana	Wati y Nugrahantoro (2026) acotan el agente para que rechace temas fuera de dominio; Andrade Andrade (2025) deriva la conversación a un asesor humano; Ghosh y Mittal (2025) construyen una consulta con recuperación de contexto en n8n (prepublicación sin revisión por pares).	RF-12: derivación a una persona cuando el flujo no pueda resolver.
Patrón disparador → enrutamiento → datos	Pandey y Lavanya (2025) y Widiyanti et al. (2026) usan Telegram con n8n; Fuady y Tundo (2025) y Siregar et al. (2026) combinan n8n con servicios de inteligencia artificial en aplicaciones web y de recomendación.	Los flujos WF-01 a WF-08 siguen el mismo patrón, independiente del canal.
Ecosistema y plantillas públicas	Tang et al. (2026) caracterizan 6 003 flujos públicos y Wu et al. (2026), 10 803 plantillas; Yu et al. (2025) revisan sistemas de flujos agentivos; Vinothraj et al. (2026) generan flujos desde lenguaje natural con n8n en Docker y los validan antes de desplegarlos; Wang et al. (2025) convierten normas de diagnóstico en un flujo ejecutable.	Versiones fijadas; las plantillas públicas se revisan antes de importarlas y los flujos se prueban antes de activarlos.

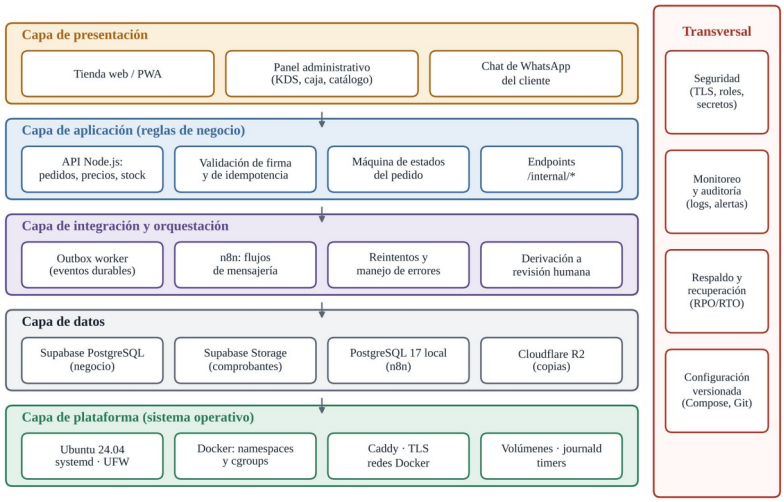
Decisión de diseño	Evidencia en la literatura	Aplicación en este diseño
Metas de desempeño propias	Venkiteela (2025) y Chen et al. (2026) miden CPU, memoria y latencia (3,22 s extremo a extremo en Chen et al.); Ruhallah et al. (2025) reportan respuestas por debajo de 3 s; Wali et al. (2025) y Amir y Atif (2026) informan mejoras muy altas, pero autoinformadas y con muestras pequeñas.	Webhook p95 < 1 s medido en PT-16; no se adoptan porcentajes de mejora de otros contextos.
Evaluación multicriterio y validación	Moreno Murillo et al. (2026) comparan alternativas con AHP-TOPSIS; Chen et al. (2026) usan AHP, costo total de propiedad a 36 meses y FMEA; Ureta Ureta (2025) presenta una propuesta sin implementación demostrada.	Matriz ponderada de alternativas (sección 5.5) y plan de pruebas, porque una propuesta no equivale a una implementación.
Indicadores de éxito	Andaluz Freire y Ulloa Gavilanes (2026) miden satisfacción con 200 clientes (CSAT 4,40); Jondec Delgado et al. (2026) obtienen SUS 81,2 con 50 estudiantes; Nguyen et al. (2025) estudian qué determina la satisfacción con agentes conversacionales; Nuñez Cartolin (2021) mide tiempos de espera.	Metas de tiempo, efectividad y satisfacción (RNF-06 y RNF-12).
Privacidad y gobernanza	Gomez-Maguiña y Sierra-Liñan (2024) y Zaga Palomino y Chire Vilcarima (2025) revisan sistemáticamente la IA en procesos de negocio; Vishwakarma et al. (2025) identifican la privacidad, la interoperabilidad y la gobernanza como barreras.	Datos mínimos del cliente y IA externa fuera del MVP.

Nota. Elaboración propia a partir de los 40 documentos de la I Unidad. Los resultados citados no se extrapolan: provienen de contextos y muestras distintos.

5.2 Arquitectura lógica

La solución se organiza en cinco capas más un conjunto de funciones transversales (Figura 1). La capa de **presentación** reúne la tienda web/PWA, el panel administrativo y el chat de WhatsApp. La capa de **aplicación** es la API de negocio, única autoridad sobre precios, stock, estados y pagos. La capa de **integración y orquestación** contiene el *outbox worker* y n8n, responsables de entregar eventos y ejecutar los flujos de mensajería. La capa de **datos** combina Supabase (negocio y archivos), un PostgreSQL local exclusivo de n8n y las copias en Cloudflare R2. La capa de **plataforma** es el sistema operativo con Docker, Caddy y las redes.

Figura 1
Arquitectura lógica por capas de RETEQUE-OS



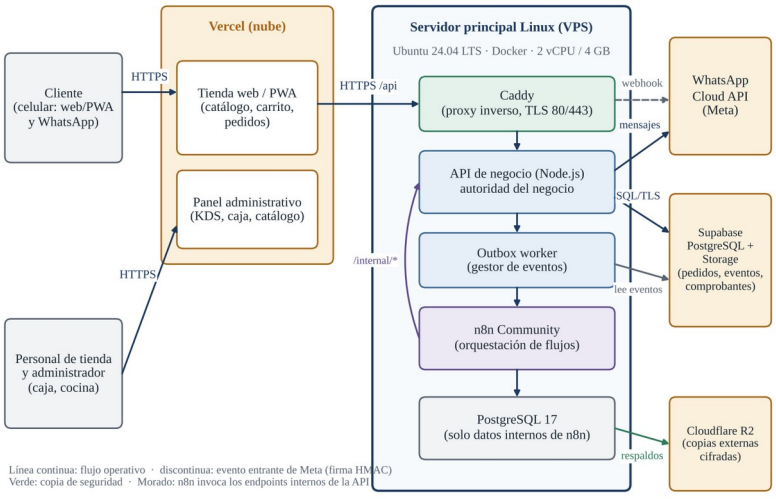
Nota. Elaboración propia a partir del documento de arquitectura (Equipo RETEQUE-OS, 2026).

El diseño adopta cinco principios: (1) **n8n coordina, la API decide**: n8n no escribe en la base de negocio y solo actúa mediante endpoints internos; (2) el webhook de Meta llega a la API, no a n8n; (3) el evento se registra en la misma transacción que el pedido; (4) comprobante recibido no equivale a pago confirmado; (5) cada capa solo se comunica con la inmediata inferior y con las funciones transversales de seguridad, monitoreo, respaldo y configuración versionada.

5.3 Arquitectura general y arquitectura física

La Figura 2 muestra la arquitectura general: los clientes y el personal acceden a la tienda y al panel alojados en Vercel; estos consumen la API que corre en la VPS detrás de Caddy; la API persiste en Supabase y envía mensajes a la WhatsApp Cloud API; el *outbox worker* lee los eventos pendientes y los entrega a n8n; n8n devuelve las acciones a la API mediante los endpoints /internal/*; y los respaldos viajan cifrados hacia Cloudflare R2.

Figura 2
Diagrama de arquitectura general



Nota. Elaboración propia a partir de la arquitectura propuesta.

La arquitectura física (Figura 5, incluida al inicio de la sección 6) corresponde a un único nodo: un Droplet de DigitalOcean con Ubuntu Server 24.04.5 LTS. En el sistema

operativo se ejecutan los servicios del anfitrión (cortafuegos, WireGuard, sshd, Docker Engine, temporizadores de respaldo y agente de monitoreo) y, sobre Docker, cinco contenedores repartidos en tres redes puente definidas por el usuario. Solo Caddy publica puertos hacia Internet; el editor de n8n se publica únicamente en la dirección de la interfaz WireGuard (10.99.0.1:5678), y PostgreSQL no publica ningún puerto.

5.4 Componentes principales

La Tabla 4 lista los componentes del sistema con su versión de referencia y su responsabilidad.

Tabla 4

Componentes principales, versiones de referencia y responsabilidades

Componente	Versión de ref.	Responsabilidad	Ubicación y exposición
Ubuntu Server	24.04.5 LTS	Sistema operativo del anfitrión: procesos (systemd), usuarios, red, cortafuegos y cgroups.	Droplet; SSH solo por wg0
Docker Engine + Compose	29.9.2	Virtualización por contenedores, redes y volúmenes; reinicio automático.	Anfitrión; sin socket expuesto
Caddy	2.11.7	Proxy inverso, certificados TLS automáticos y cabeceras de seguridad (HSTS).	Contenedor; 80/443 públicos
API de negocio	Node.js 24 LTS	Webhook de Meta, reglas, estados, pagos, endpoints /internal/*, autorización y auditoría.	Contenedor; vía Caddy
Outbox worker	Node.js 24 LTS	Lee eventos pendientes y los entrega a n8n con reintentos y espera exponencial.	Contenedor; interno
n8n Community	2.45.7	Orquestación de los flujos WF-01 a WF-08 (mensajes, cierre, pago, derivación y errores).	Contenedor; editor solo por wg0
PostgreSQL	17.11	Datos internos de n8n (flujos, credenciales cifradas y ejecuciones).	Contenedor; red interna
Supabase	Servicio gestionado	PostgreSQL de negocio (pedidos, eventos, usuarios) y Storage de comprobantes.	Externo; TLS
WhatsApp Cloud API	Servicio de Meta	Canal oficial de mensajes y webhooks.	Externo; HTTPS
WireGuard	Módulo del núcleo	VPN administrativa (10.99.0.0/24) para SSH y editor de n8n.	Anfitrión; 51820/udp
restic + pg_dump	Estable vigente	Copias cifradas y deduplicadas hacia Cloudflare R2; volcado lógico de las bases.	Anfitrión; salida 443

Nota. Versiones de referencia definidas por el equipo. Se fijarán las etiquetas exactas y se confirmará su vigencia en las páginas oficiales antes del despliegue.

5.5 Decisiones de arquitectura y alternativas descartadas

Cada decisión se contrastó con al menos una alternativa. La Tabla 5 resume las alternativas descartadas y el criterio decisivo, y la Tabla 6 compara cuatro formas de desplegar la solución con una matriz ponderada.

Tabla 5
Decisiones de arquitectura y alternativas descartadas

Decisión	Alternativas descartadas	Criterio decisivo
Plataforma de orquestación: n8n Community autoalojado	Make y Zapier (servicios cerrados); Node-RED, Airflow y Boomi (Venkiteela, 2025); n8n Cloud	Edición Community autoalojable, sin costo de licencia por uso interno y con control de los datos (Moreno, 2025; Flores Chino et al., 2026).
Aislamiento: contenedores Docker	Máquinas virtuales completas; instalación directa en el sistema operativo; Kubernetes	Aislamiento por *namespaces* y cgroups con menos memoria que un hipervisor (Bui, 2015); un solo nodo de 4 GB no justifica un orquestador de clústeres.
Alojamiento: Droplet de DigitalOcean	Servidor físico propio; n8n Cloud con servicios cerrados; clúster gestionado	Mayor puntaje en la matriz ponderada (Tabla 6); Chen et al. (2026) estiman ≈ US$ 1 640 de hardware para un servidor local.
Proxy inverso: Caddy	Nginx; Traefik	Certificados TLS automáticos con configuración mínima; es el proxy del caso de Muñoz Ingaruca (2026).
Acceso remoto: WireGuard y SSH con claves	OpenVPN; SSH expuesto con fail2ban	El puerto 51820/udp no responde a paquetes no autenticados y el código del protocolo es pequeño (Donenfeld, 2017); SSH deja de estar en Internet.
Respaldo: restic hacia R2 y copia del proveedor	rsync a otro disco; solo instantáneas del proveedor	Copias cifradas y deduplicadas fuera del servidor y del proveedor de la VPS (regla 3-2-1, sección 8.4).
Escalado: contenedor único en el MVP	Modo cola con Redis y workers desde el inicio	Venkiteela (2025) reporta 5,1 GB de memoria con 100 trabajos simultáneos; con ≈ 270 mensajes diarios no hace falta. Se activa al superar los umbrales de RNF-08.
Monitoreo: alertas del proveedor, sonda externa y registros locales	SIEM completo (pila ELK o Wazuh)	Un SIEM consume memoria que la VPS de 4 GB no tiene; se difiere hasta que crezca el volumen o el servidor.
Inteligencia artificial fuera del MVP	Agente con autoridad sobre precios, estados y pagos	Riesgo de inyección indirecta (Fendley et al., 2026) y menor desempeño en el seguimiento del estado (Hudeček y Dušek, 2023).

Nota. Elaboración propia a partir de la revisión de la I Unidad.

Tabla 6
Matriz ponderada de alternativas de despliegue

Criterio	Peso	A. VPS y Docker	B. Servidor propio	C. n8n Cloud y SaaS	D. Kubernetes gestionado
Costo recurrente	25 %	4	2	3	1
Control del SO y aislamiento	20 %	5	5	1	4

Criterio	Peso	A. VPS y Docker	B. Servidor propio	C. n8n Cloud y SaaS	D. Kubernetes gestionado
Seguridad y operación	20 %	4	3	3	4
Rapidez de implementación (8 semanas)	15 %	4	2	5	1
Escalabilidad	10 %	3	2	4	5
Independencia del proveedor	10 %	3	5	1	3
Puntaje ponderado (1 a 5)	100 %	4,00	3,10	2,80	2,80

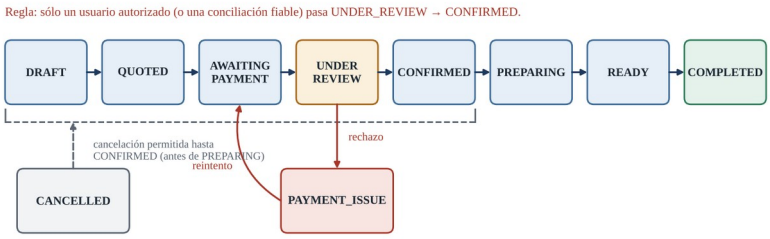
Nota. Elaboración propia. Los puntajes de 1 (peor) a 5 (mejor) son un juicio del equipo y no una medición. Con pesos iguales, A obtiene 3,83 frente a 3,17, 2,83 y 3,00; con 10 000 combinaciones aleatorias de pesos, A resulta primera en el 92,8 % de los casos.

La alternativa A resulta la mejor combinación entre costo, control del sistema operativo y rapidez de implementación; sus puntos débiles son la escalabilidad vertical y la dependencia del proveedor del servidor, que se mitigan con el respaldo externo y la ruta de crecimiento del requerimiento RNF-08.

5.6 Flujo de información

El estado del pedido se controla con una máquina de estados (Figura 3). Los estados del pedido y del pago se guardan por separado, y cada transición exige la versión esperada del registro para evitar condiciones de carrera entre operadores.

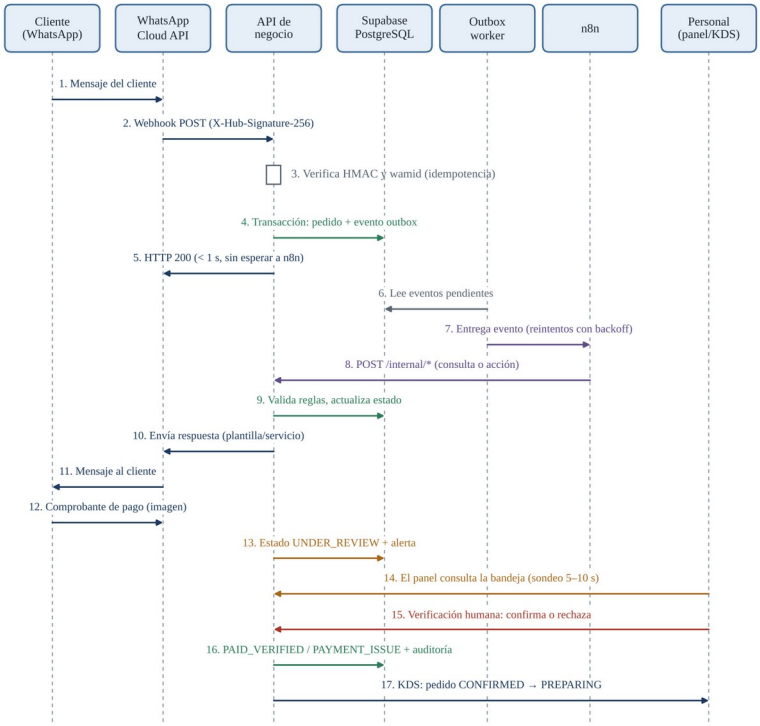
Figura 3
Máquina de estados del pedido



Nota. Elaboración propia a partir del documento de arquitectura (Equipo RETEQUE-OS, 2026).

La Figura 4 detalla el recorrido de un pedido, desde el mensaje del cliente hasta la preparación en cocina. Los pasos 1 a 5 ocurren de forma síncrona y deben completarse en menos de un segundo; la entrega a n8n (pasos 6 a 11) es asíncrona y reintentable, de modo que una caída de n8n no pierde el pedido: el evento permanece en la tabla *outbox* hasta ser entregado. Los pasos 12 a 17 implementan la regla de verificación humana del pago.

Figura 4
Flujo de información de un pedido por WhatsApp (secuencia)



Nota. Elaboración propia a partir de las secciones de flujo y confiabilidad del documento de arquitectura (Equipo RETEQUE-OS, 2026).

Los canales de datos entre componentes, con su protocolo y su autenticación, se resumen en la Tabla 7.

Tabla 7
Canales de datos del sistema

Flujo	Origen → destino	Protocolo y puerto	Autenticación	Datos
Cliente → tienda/panel	Navegador → Vercel	HTTPS 443	Sesión y roles	Catálogo, carrito, pedidos
Tienda/panel → API	Vercel → Caddy → API	HTTPS 443	Token de usuario	Pedidos, estados
Meta → API	WhatsApp Cloud API → Caddy → API	HTTPS 443	Token de verificación y firma HMAC	Mensajes, multimedia
API → Supabase	API → PostgreSQL/Storage	TLS 5432/443	Credencial de servicio	Pedidos, eventos, archivos
Outbox → n8n	Worker → n8n	HTTP interno 5678 (net_app)	Secreto compartido	Eventos
n8n → API	n8n → API /internal/*	HTTP interno 3000 (net_app)	Secreto compartido	Consultas y acciones
API → Meta	API → WhatsApp Cloud API	HTTPS 443 saliente	Token de acceso de Meta	Mensajes de salida

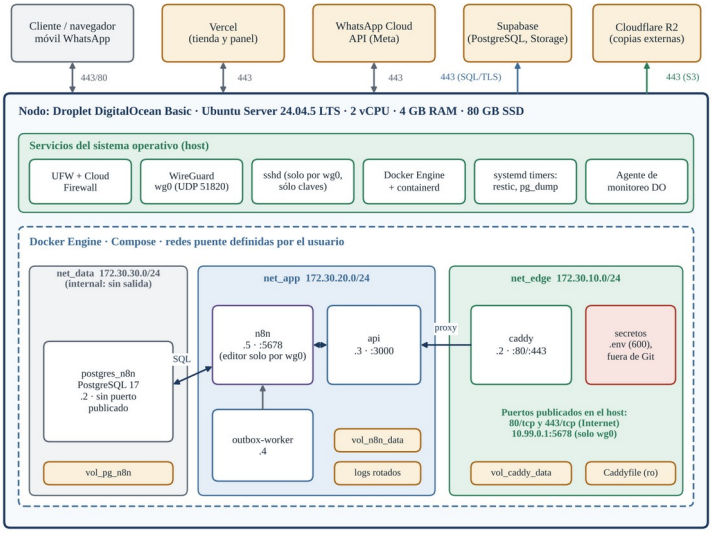
Flujo	Origen → destino	Protocolo y puerto	Autenticación	Datos
Respaldos	Anfitrión → Cloudflare R2	HTTPS 443 saliente	Credencial limitada + cifrado	Copias cifradas

Nota. Elaboración propia. Los secretos compartidos y credenciales se almacenan fuera del repositorio (sección 8).

6. DISEÑO DE INFRAESTRUCTURA

Esta sección define los elementos de infraestructura que soportan la arquitectura de la sección 5. La Figura 5 presenta el diagrama de despliegue del servidor principal y sirve de referencia para las subsecciones siguientes, que detallan los servidores, los equipos, el sistema operativo, el almacenamiento, la virtualización y los servicios requeridos.

Figura 5
Diagrama de despliegue del servidor principal



Nota. Elaboración propia. Las direcciones IP son de diseño; la IP pública 203.0.113.10 es un rango de documentación (RFC 5737) y se sustituirá por la asignada por el proveedor.

6.1 Servidores

La infraestructura propia se reduce a un servidor virtual privado, por lo que no se requiere servidor físico. El resto de los servicios se contratan como plataforma gestionada: Vercel para la tienda y el panel, Supabase para la base de negocio y los archivos, y Cloudflare R2 para las copias externas.

La Tabla 8 especifica el servidor y los servicios contratados, con su justificación.

Tabla 8
Especificación de los servidores y servicios de infraestructura

Elemento	Especificación	Justificación
VPS principal	DigitalOcean Basic: 2 vCPU, 4 GB de RAM, 80 GB SSD, 4 TB de transferencia; Ubuntu Server 24.04.5 LTS; copia diaria del servidor activada.	Coincide con el diseño de Muñoz Ingaruca (2026) y deja margen para n8n, la API y PostgreSQL.
Región del Droplet	Por definir en la III Unidad: se medirá la latencia hacia Tacna para cumplir RNF-05.	Muñoz Ingaruca (2026) usó NYC3 con ≈ 150 ms hacia Perú.

Elemento	Especificación	Justificación
IP pública	Una dirección IPv4 pública (preferible reservada) asociada a los registros DNS del dominio.	Evita cambios de DNS al reconstruir el servidor.
Vercel	Plan comercial para tienda web/PWA y panel (S/ 69,34/mes, ≈ US$ 20).	El plan Hobby es de uso no comercial.
Supabase	PostgreSQL gestionado y Storage (S/ 86,68/mes).	Delega la base de negocio y los archivos.
Cloudflare R2	Bucket privado para copias externas (hasta 10 GB sin costo).	Almacenamiento fuera del servidor y del proveedor de la VPS.

Nota. Elaboración propia. Los precios son referenciales.

6.2 Equipos

En el local se utilizarán equipos ya existentes: una computadora del administrador para el panel y la VPN, un equipo o tablet para la pantalla de cocina (KDS) y la conexión a Internet del negocio. Para aplicar la segmentación de la sección 7 se recomienda un router/cortafuegos con soporte VLAN y un switch administrable; estos equipos son opcionales, no están en el presupuesto base y su cotización se deja pendiente (sección 12.2). Mientras no existan, la red local funcionará como un único segmento y la protección dependerá de los controles de la VPS.

6.3 Sistemas operativos

El anfitrión usa Ubuntu Server 24.04.5 LTS por su soporte de largo plazo y por ser la plataforma de referencia de Docker y WireGuard. La configuración base del sistema operativo incluye:

- **Cuentas y acceso:** usuario administrador sin contraseña de SSH, ingreso con clave pública, inicio de sesión de root deshabilitado, `sudo` con registro y bloqueo de intentos repetidos (fail2ban).
- **Actualizaciones:** parches de seguridad automáticos (`unattended-upgrades`) y ventana mensual para el resto de paquetes.
- **Núcleo y memoria:** swap de 2 GB con `vm.swappiness=10`, sincronización horaria (chrony) y zona horaria America/Lima.
- **Servicios y registros:** unidades systemd con reinicio ante fallos; límite de tamaño del journal y rotación de registros de Docker.
- **Auditoría:** `auditd` con reglas mínimas sobre /etc, claves SSH y el archivo de entorno.

6.4 Almacenamiento

El disco de 80 GB se reparte según la Tabla 9.

Tabla 9

Presupuesto de almacenamiento del disco de 80 GB

Uso	Estimado	Política
Sistema operativo y paquetes	10 GB	Sin servicios innecesarios instalados.
Imágenes y capas de Docker	8 GB	Limpieza de imágenes no usadas tras cada actualización.
Volumen PostgreSQL de n8n	5 GB	Poda de ejecuciones antiguas (p. ej. 7 días).

Uso	Estimado	Política
Volumen de datos de n8n y Caddy	2 GB	Incluye certificados TLS.
Registros (journal y contenedores)	3 GB	Rotación y compresión.
Área temporal de respaldos	10 GB	Se limpia tras subir a R2.
Swap	2 GB	Archivo de intercambio.
Libre (reserva)	40 GB	Alerta al 80 % de uso.

Nota. Elaboración propia. Las cifras son estimaciones de planificación que se contrastarán con df y docker system df en las pruebas.

Los datos del negocio y los comprobantes no residen en el disco de la VPS sino en Supabase; en la VPS solo permanecen los datos de n8n y la configuración. Los volúmenes de Docker son persistentes y se montan con permisos mínimos (la configuración de Caddy en solo lectura). El cifrado en reposo de las copias lo aplica restic; el disco del Droplet no se cifra por completo, por lo que ningún secreto se guarda en texto claro fuera del archivo de entorno con permisos 600.

6.5 Virtualización

La virtualización es por contenedores del sistema operativo, no por máquinas virtuales completas: los contenedores comparten el núcleo de Ubuntu y se aíslan mediante *namespaces* (Bui, 2015), lo que consume menos memoria que un hipervisor, un factor crítico en un servidor de 4 GB. Cada contenedor recibe su propio espacio de procesos (PID), de red, de montaje, de comunicación entre procesos (IPC) y de nombre de host, y sus recursos se limitan con *cgroups*. La Tabla 10 propone los límites iniciales.

Tabla 10
Límites de recursos por contenedor (cgroups) y consumo estimado

Servicio	Memoria máx.	CPU máx.	Consumo estimado	Observación
n8n	1 280 MB	1,5	400–800 MB	Mayor consumidor; límite evita que agote el servidor.
postgres (n8n)	640 MB	1,0	200–400 MB	Memoria compartida acotada.
api	512 MB	1,0	150–250 MB	Node.js con límite de heap.
outbox-worker	256 MB	0,5	80–150 MB	Proceso de sondeo liviano.
caddy	96 MB	0,5	20–40 MB	Proxy inverso.
Suma de límites	2 784 MB	—	—	Más ≈ 0,7 GB del sistema operativo y Docker: < 4 GB; swap de reserva.

Nota. Elaboración propia. El consumo es una estimación de planificación; Venkiteela (2025) reportó 2,3 a 5,1 GB de memoria en su banco de pruebas con 10 a 100 trabajos simultáneos, por lo que se vigilará el margen real en PT-17.

Los contenedores se endurecen con las siguientes opciones de Compose (fragmento representativo para n8n; el Anexo A reúne otras configuraciones de referencia):

```
n8n:
  image: n8nio/n8n:<etiqueta-fija>      # sin 'latest'
  user: "1000:1000"                      # no root
  restart: unless-stopped
  security_opt: ["no-new-privileges:true"]
  cap_drop: ["ALL"]
  mem_limit: 1280m
  cpus: 1.5
  healthcheck: { test:
["CMD","wget","-qO-","http://localhost:5678/healthz"] }
  networks: [net_app, net_data]
  ports: ["10.99.0.1:5678:5678"]         # solo wg0 (Docker tras wg-
quick@wg0)
  env_file: .env                         # permisos 600, fuera de Git
```

6.6 Servicios requeridos

La Tabla 11 resume los servicios requeridos y su política de arranque y recuperación.

Tabla 11

Servicios requeridos en el servidor y servicios externos

Servicio	Tipo	Función	Arranque y recuperación
docker.service	Anfitrión	Motor de contenedores y redes.	Habilitado en el arranque
Contenedores (5)	Docker	caddy, api, outbox-worker, n8n, postgres.	restart: unless-stopped y healthchecks
wg-quick@wg0	Anfitrión	VPN administrativa.	Habilitado en el arranque
ssh.service	Anfitrión	Administración remota (solo por wg0).	Habilitado
ufw.service	Anfitrión	Cortafuegos del anfitrión.	Habilitado
fail2ban, auditd, chrony	Anfitrión	Bloqueo de fuerza bruta, auditoría y hora.	Habilitados
Temporizadores systemd	Anfitrión	Respaldo diario (02:00) y verificación semanal.	Persistentes ante reinicios
Agente de monitoreo	Proveedor	Métricas y alertas de CPU, RAM y disco.	Gestionado
Supabase, Meta, R2, Vercel	Externo	Datos, mensajería, copias y frontend.	SLA del proveedor

Nota. Elaboración propia.

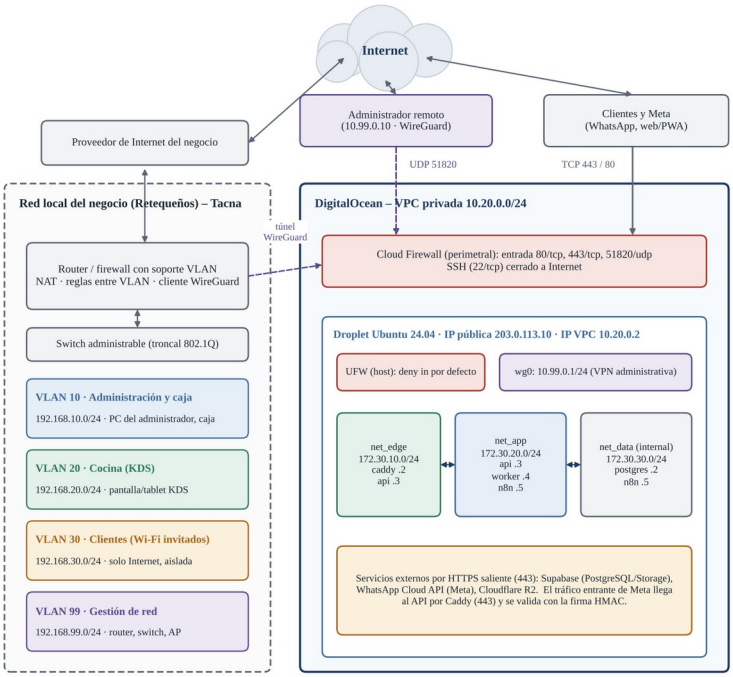
7. DISEÑO DE RED

7.1 Topología

La red tiene dos ámbitos conectados por Internet: la **red local del negocio** (administración, cocina y clientes) y la **red de la VPS** en la nube. La Figura 6 muestra la topología. Los clientes y Meta llegan al servidor por los puertos 80 y 443; el administrador remoto y la sede se conectan por un túnel WireGuard al puerto 51820/udp; y el resto de los

puertos permanece cerrado. Dentro de la VPS, el tráfico se reparte en tres redes Docker con reglas distintas.

Figura 6
Topología de red de RETEQUE-OS



Nota. Elaboración propia. Las direcciones son de diseño; 203.0.113.10 pertenece al rango de documentación (RFC 5737).

7.2 Segmentación

La Tabla 12 define qué zonas pueden iniciar conexiones y cuáles no.

Tabla 12
Zonas de red y reglas de comunicación

Zona	Componentes	Puede iniciar conexión hacia	No puede
Internet	Clientes, Meta, administrador	Solo 80/443 y 51820/udp de la VPS	Acceder a SSH, n8n, PostgreSQL o la API interna
net_edge	caddy, api	net_app (API)	Alcanzar PostgreSQL
net_app	api, outbox-worker, n8n	Supabase, Meta, R2 (443 saliente); net_data (n8n)	Recibir tráfico directo de Internet
net_data (interna)	postgres_n8n, n8n	Ninguna conexión saliente	Salir a Internet; ser alcanzada por caddy
VLAN 10	Administración y caja	VPS por HTTPS y WireGuard; Internet	Acceder a VLAN 20, 30 y 99 salvo lo permitido
VLAN 20	Cocina (KDS)	Panel por HTTPS	Acceder a VLAN 10, 30 y 99
VLAN 30	Clientes (Wi-Fi invitados)	Solo Internet	Acceder a cualquier otra VLAN
VLAN 99	Gestión de red	Actualizaciones del fabricante	Recibir tráfico desde VLAN 30

Nota. Elaboración propia. En la VPS la segmentación se logra con redes Docker, la VPC y los cortafuegos; las VLAN se aplican solo a la red local.

7.3 Direccionamiento IP

La Tabla 13 asigna a cada segmento un bloque de direcciones privadas del rango definido en RFC 1918 (Rekhter et al., 1996); la dirección pública 203.0.113.10 pertenece al bloque de documentación reservado en RFC 5737 (Arkko et al., 2010). La ausencia de solapamientos se comprueba en la sección 11.4.

Tabla 13
Plan de direccionamiento IP

Segmento	Red (CIDR)	Puerta	Asignación
IP pública del Droplet	203.0.113.10/32 (doc.)	—	Reservada; registros DNS A de la tienda, la API y el panel
VPC de DigitalOcean	10.20.0.0/24	10.20.0.1	Droplet: 10.20.0.2; reservado para futuras réplicas
VPN WireGuard (wg0)	10.99.0.0/24	—	Servidor 10.99.0.1; administradores 10.99.0.10–19; router de sede 10.99.0.2
Docker net_edge	172.30.10.0/24	172.30.10.1	caddy .2; api .3
Docker net_app	172.30.20.0/24	172.30.20.1	api .3; outbox-worker .4; n8n .5
Docker net_data	172.30.30.0/24 (interna)	172.30.30.1	postgres_n8n .2; n8n .5
VLAN 10 Administración	192.168.10.0/24	192.168.10.1	DHCP .100–.199; reservas para caja y administrador
VLAN 20 Cocina	192.168.20.0/24	192.168.20.1	KDS con reserva .10
VLAN 30 Clientes	192.168.30.0/24	192.168.30.1	DHCP .50–.250; aislamiento de clientes
VLAN 99 Gestión	192.168.99.0/24	192.168.99.1	Router .1; switch .2; punto de acceso .3

Nota. Elaboración propia. Las redes 172.30.x.x se fijan en Compose para evitar solapes con el rango 172.17.0.0/16 que Docker usa por defecto.

7.4 VLAN

Las VLAN definidas en IEEE 802.1Q (Institute of Electrical and Electronics Engineers [IEEE], 2022) segmentan la red local del negocio (Tabla 14): administración y caja, cocina, clientes y gestión, de modo que el Wi-Fi de invitados no pueda alcanzar los equipos de caja ni la pantalla de cocina. En una VPS de nube el cliente no controla la capa 2 del proveedor, por lo que la segmentación equivalente se obtiene con la VPC, las redes Docker y los cortafuegos descritos arriba. La implementación de las VLAN dependerá de contar con un router y un switch compatibles; si no están disponibles en la III Unidad, se validará el diseño en un laboratorio virtual (por ejemplo, interfaces VLAN de Linux o un simulador de redes) y se documentará como pendiente de despliegue físico.

Tabla 14
Definición de las VLAN de la red local

VLAN	Nombre	Subred	Dispositivos	Política
10	Administración y caja	192.168.10.0/24	PC del administrador, caja	Internet y VPN; acceso al panel
20	Cocina (KDS)	192.168.20.0/24	Pantalla o tablet del KDS	Solo panel por HTTPS
30	Clientes	192.168.30.0/24	Wi-Fi de invitados	Solo Internet; aislamiento de clientes
99	Gestión	192.168.99.0/24	Router, switch y punto de acceso	Solo administración

Nota. Elaboración propia. Troncal 802.1Q entre router y switch; puertos de acceso por VLAN.

7.5 Firewall

Se aplican tres niveles de filtrado: el cortafuegos de nube del proveedor (perímetro), UFW en el anfitrión y las reglas de la cadena DOCKER-USER. Esto responde a una particularidad de Docker: al publicar un puerto, Docker inserta reglas de iptables propias que pueden eludir a UFW, por lo que únicamente se publican los puertos necesarios y el control se refuerza en el perímetro.

La Tabla 15 concreta las reglas de cada nivel de filtrado, con su sentido y su motivo.

Tabla 15
Reglas de cortafuegos propuestas

Nivel	Sentido	Regla	Motivo
Cloud Firewall	Entrada	Permitir 80/tcp y 443/tcp desde cualquier origen	Tienda, API y webhook de Meta; el puerto 80 solo redirige a HTTPS
Cloud Firewall	Entrada	Permitir 51820/udp	VPN administrativa (WireGuard no responde a paquetes no autenticados)
Cloud Firewall	Entrada	Denegar el resto, incluido 22/tcp	SSH no se expone a Internet
Cloud Firewall	Salida	Permitir 443/tcp, 53 y 123/udp	Supabase, Meta, R2, DNS y hora
UFW (host)	Entrada	deny por defecto; allow 80, 443, 51820/udp; allow 22 solo desde wg0	Defensa en profundidad si falla el perímetro
DOCKER-USER	Reenvío	Permitir solo tráfico a caddy desde Internet; bloquear el resto	Evitar que los puertos publicados eludan UFW
net_data	Salida	Red Docker con internal: true	La base de n8n no tiene ruta a Internet

Nivel	Sentido	Regla	Motivo
Router de sede	Entre VLAN	Denegar VLAN 30 → todas; permitir VLAN 10 → VLAN 20 (panel) y VLAN 10/20 → Internet	Aislar a los clientes

Nota. Elaboración propia a partir del diseño de seguridad del proyecto.

7.6 Acceso remoto

- **VPN WireGuard (Donenfeld, 2017):** único camino para administrar el servidor. Cada administrador tiene un par de claves propio; la revocación consiste en eliminar su clave pública.
- **SSH:** solo escucha en la interfaz wg0, con autenticación por clave pública, sin contraseñas y sin acceso de root.
- **Editor de n8n:** accesible en 10.99.0.1:5678 desde la VPN, con usuario y contraseña propios y, si la versión lo permite, doble factor. No existe una ruta pública hacia el editor.
- **Paneles web:** acceso por HTTPS con roles y autenticación de múltiples factores para administradores.
- **Cuentas de proveedores:** autenticación de múltiples factores obligatoria en DigitalOcean, Cloudflare, Supabase, Vercel, GitHub y Meta Business.
- **Recuperación de acceso:** la consola de recuperación del proveedor actúa como vía de emergencia si la VPN falla; su uso queda registrado.

8. DISEÑO DE SEGURIDAD

El diseño de seguridad se basa en tres principios: defensa en profundidad, mínimo privilegio y denegación por defecto. Parte de los riesgos de la evaluación inicial del equipo, de los controles del documento de arquitectura (Equipo RETEQUE-OS, 2026) y de la literatura sobre aislamiento de contenedores (Bui, 2015) y sobre riesgos de los flujos con agentes (Tang et al., 2026; Fendley et al., 2026). La gestión de riesgos sigue las etapas de ISO/IEC 27005:2022 (identificación, análisis, evaluación y tratamiento), los controles se asocian al Anexo A de ISO/IEC 27001:2022 y las funciones de seguridad se alinean con NIST CSF 2.0.

8.1 Riesgos identificados

Cada riesgo se valora con probabilidad (P) e impacto (I) en una escala de 1 a 5, como propone ISO/IEC 27005:2022 para el análisis cualitativo (ISO/IEC, 2022b); el nivel resulta del producto (≥ 15 alto; 8 a 14 medio; < 8 bajo). La valoración es una propuesta del equipo y se revisará con los resultados de las pruebas.

La Tabla 16 presenta los doce riesgos identificados con su valoración.

Tabla 16
Matriz de riesgos

ID	Riesgo	P	I	P×I	Nivel
R-01	Caída de Internet en el negocio o en el proveedor de la VPS.	3	3	9	Medio
R-02	Sobrecarga o agotamiento de CPU, memoria o disco del servidor.	3	4	12	Medio

ID	Riesgo	P	I	P×I	Nivel
R-03	Fallo en el envío de mensajes de WhatsApp (límites, plantillas, caída de Meta).	3	3	9	Medio
R-04	Pérdida de datos por falla de disco, borrado accidental o corrupción.	2	5	10	Medio
R-05	Acceso no autorizado al servidor o a los paneles (fuerza bruta, credenciales robadas).	3	5	15	Alto
R-06	Webhook falsificado o mensajes repetidos que generen pedidos duplicados.	3	4	12	Medio
R-07	Exposición accidental del editor de n8n o de PostgreSQL por configuración de red.	2	5	10	Medio
R-08	Fuga o pérdida de secretos (tokens, .env, llave de cifrado de n8n).	2	5	10	Medio
R-09	Abuso de contenedores por privilegios excesivos o socket de Docker expuesto.	1	5	5	Bajo
R-10	Comprobante falso o alterado aceptado como pago.	3	4	12	Medio
R-11	Inyección indirecta de instrucciones si se incorpora IA al flujo.	2	4	8	Medio
R-12	Vulnerabilidades en n8n o en imágenes por software desactualizado.	3	4	12	Medio

Nota. Elaboración propia. R-01 a R-05 son los riesgos iniciales del equipo; R-06 a R-12 se derivaron del documento de arquitectura y de la literatura.

8.2 Amenazas y controles propuestos

La Tabla 17 relaciona cada riesgo con la amenaza (clasificada según el modelo STRIDE), los controles de diseño, el control del Anexo A de ISO/IEC 27001:2022 (ISO/IEC, 2022a) y la prueba que los verifica. La Figura 7 muestra cómo los controles se distribuyen en zonas de confianza, desde el perímetro hasta los datos y el aislamiento en el sistema operativo.

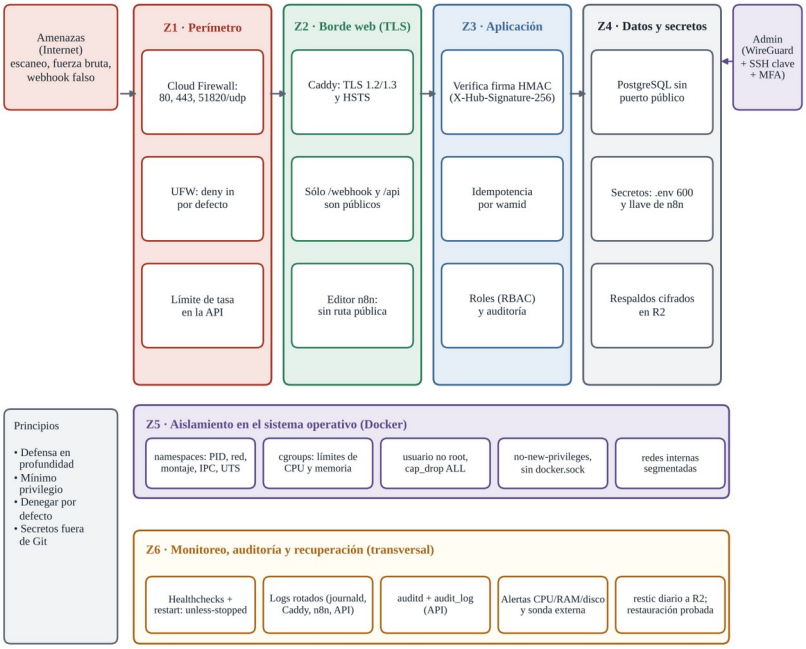
Tabla 17
Amenazas, controles y verificación

Riesgo	Amenaza (STRIDE)	Controles propuestos	ISO/IEC 27001:2022 (Anexo A)	Prueba
R-01	Denegación de servicio por caída de la red del negocio o del proveedor.	Proveedor con acuerdo de nivel de servicio; sonda externa; reintentos del outbox; contingencia manual en caja hasta reconectar.	A.5.30 y A.8.14	PT-06, PT-24
R-02	Denegación de servicio por agotamiento de recursos.	Límites de CPU y memoria (cgroups); limitación de tasa; poda de datos; alertas; plan de ampliación.	A.8.6 y A.8.16	PT-04, PT-17
R-03	Fallo de una dependencia externa (mensajería de Meta).	Reintentos con espera exponencial; plantillas aprobadas; alerta si los fallos superan 5 %; aviso manual al cliente.	A.5.23 y A.8.16	PT-23, PT-24

Riesgo	Amenaza (STRIDE)	Controles propuestos	ISO/IEC 27001:2022 (Anexo A)	Prueba
R-04	Destrucción o pérdida de datos.	Copia diaria cifrada fuera del servidor; copia del proveedor; restauración probada.	A.8.13 y A.5.30	PT-19, PT-20
R-05	Suplantación y elevación de privilegios por fuerza bruta o credenciales robadas.	SSH solo por VPN y con claves; fail2ban; MFA en paneles; contraseñas únicas; cuentas por persona; revocación inmediata.	A.5.15, A.5.17 y A.8.5	PT-09
R-06	Suplantación del remitente; repetición de mensajes.	Verificación del token (GET) y de la firma HMAC X-Hub-Signature-256 sobre el cuerpo original; restricción única por wamid; limitación de tasa en la API.	A.8.26 y A.8.24	PT-11, PT-12
R-07	Divulgación de información por servicios expuestos.	Solo caddy publica puertos; editor de n8n atado a wg0; PostgreSQL sin puerto; red interna; DOCKER-USER; escaneo con nmap en cada cambio.	A.8.20 y A.8.22	PT-05, PT-08, PT-13
R-08	Divulgación de secretos.	.env con permisos 600 y fuera de Git; llave de n8n respaldada por separado; rotación cada 90 días; escaneo de secretos en el repositorio.	A.5.17 y A.8.24	PT-14
R-09	Elevación de privilegios desde un contenedor.	Sin --privileged; usuario no root; cap_drop ALL; no-new-privileges; sin montar docker.sock; imágenes oficiales con versión fija (Bui, 2015).	A.8.2 y A.8.9	PT-03
R-10	Repudio y manipulación de comprobantes.	Regla comprobante ≠ pago; estado UNDER_REVIEW; verificación humana autorizada; auditoría de quién aprueba y cuándo (Tang et al., 2026).	A.5.3 y A.8.15	PT-15
R-11	Manipulación de la entrada del agente.	IA fuera del MVP; si se activa, solo extrae datos en JSON validado por el backend y sin autoridad sobre precios, stock ni pagos (Fendley et al., 2026).	A.8.26	—
R-12	Explotación de vulnerabilidad es conocidas.	Versiones fijadas y revisadas; parches de seguridad automáticos del SO; actualización mensual con respaldo previo y reversión.	A.8.8 y A.8.32	PT-21

Nota. Elaboración propia. STRIDE: suplantación, manipulación, repudio, divulgación, denegación de servicio y elevación de privilegios. Los números A.x corresponden a ISO/IEC 27001:2022. R-11 se verificará solo si se incorpora IA.

Figura 7
Diagrama de seguridad: zonas de confianza y controles



Nota. Elaboración propia a partir de la sección de seguridad del documento de arquitectura (Equipo RETEQUE-OS, 2026).

Configuración de seguridad de n8n. Además del aislamiento, se fijan variables de entorno de endurecimiento: llave de cifrado estable (`N8N_ENCRYPTION_KEY`), bloqueo del acceso a variables de entorno desde los nodos (`N8N_BLOCK_ENV_ACCESS_IN_NODE`), exclusión de nodos de ejecución de comandos (`NODES_EXCLUDE`), poda de ejecuciones (`EXECUTIONS_DATA_PRUNE`) y telemetría desactivada. Los nombres exactos se confirmarán en la documentación de la versión fijada antes del despliegue.

Las funciones de NIST CSF 2.0 (National Institute of Standards and Technology [NIST], 2024) se alinean con el diseño según la Tabla 18.

Tabla 18
Alineación con NIST CSF 2.0

Función	Elementos del diseño	Dónde se describe
Gobernar	Roles y responsabilidades (RACI), políticas de acceso y de secretos, revisión de riesgos.	Secciones 8.3 y 9.1
Identificar	Inventario de componentes, matriz de riesgos y valoración P × I.	Secciones 5.4 y 8.1
Proteger	Aislamiento por contenedor, segmentación, cortafuegos, VPN, cifrado TLS, control de acceso por rol y respaldos.	Secciones 6, 7 y 8.2 a 8.4
Detectar	Healthchecks, umbrales de alerta, sonda externa, auditd y bitácora de auditoría.	Sección 8.5
Responder	Severidades P1 a P4, guías de respuesta y revisión posterior.	Sección 9.4
Recuperar	Restauración desde R2, RPO ≤ 24 h, RTO ≤ 2 h y prueba mensual.	Sección 8.4

Nota. Elaboración propia a partir de NIST (2024).

8.3 Gestión de accesos

El acceso se otorga por rol y con el mínimo necesario. La Tabla 19 resume los permisos del panel; n8n no tiene cuenta en la base de negocio y solo opera con las credenciales de la API interna.

Tabla 19
Matriz de roles y permisos del panel

Permiso	ADMIN	CAJA	OPERADOR	COCINA	AUDITOR	n8n (servicio)
Ver pedidos	Sí	Sí	Sí	Sí	Sí	Vía /internal
Cambiar estado de preparación	Sí	—	—	Sí	—	—
Ver comprobantes de pago	Sí	Sí	Sí	—	Sí	—
Aprobar o rechazar pagos	Sí	Sí	—	—	—	—
Editar catálogo y precios	Sí	—	—	—	—	—
Gestionar usuarios y roles	Sí	—	—	—	—	—
Ver la bitácora de auditoría	Sí	—	—	—	Sí	—
Editor de n8n (por VPN)	Sí	—	—	—	—	—

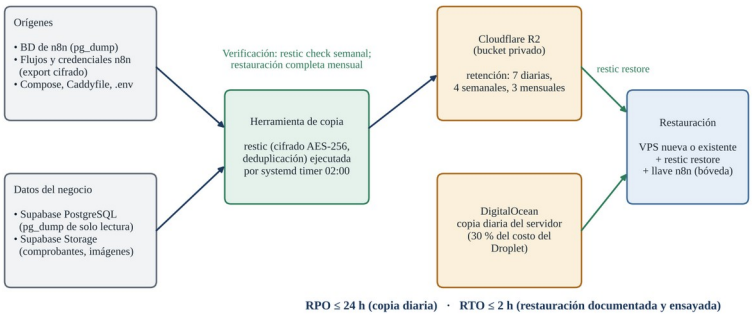
Nota. Elaboración propia a partir de los roles del documento de arquitectura (Equipo RETEQUE-OS, 2026). COCINA solo ve los pedidos ya confirmados.

- **Autenticación:** contraseñas únicas y robustas, MFA para ADMIN y CAJA, sesiones con expiración y bloqueo por intentos fallidos.
- **Cuentas de servicio:** credenciales separadas por función (API, worker, respaldo de solo lectura) con permisos mínimos; ninguna se comparte entre personas.
- **Ciclo de vida:** alta y baja documentadas; al retirarse un integrante se revoca su clave WireGuard, su clave SSH y sus sesiones el mismo día.
- **Secretos:** rotación cada 90 días y ante sospecha de fuga; la llave de cifrado de n8n se guarda en una bóveda separada de los respaldos.

8.4 Respaldo y recuperación

La estrategia combina tres capas: la copia diaria del servidor que ofrece el proveedor, las copias lógicas cifradas hacia Cloudflare R2 (Figura 8) y la copia propia de Supabase. Cumple lo definido en el diseño (copia diaria, almacenamiento externo, restauración del servicio y prueba mensual) y las metas **RPO ≤ 24 h** (la pérdida máxima es un día de datos) y **RTO ≤ 2 h** (el servicio se restablece en un máximo de dos horas).

Figura 8
Cadena de respaldo y recuperación



Nota. Elaboración propia.

La Tabla 20 detalla qué se respalda, con qué frecuencia y retención.

Tabla 20
Política de respaldo

Qué se respalda	Frecuencia	Herramienta	Destino	Retención
Base de n8n (volcado lógico)	Diaria 02:00	pg_dump + restic	R2 (cifrado)	7 diarias, 4 semanales, 3 mensuales
Flujos y credenciales de n8n (exportación cifrada)	Diaria	CLI de n8n + restic	R2 (cifrado)	Igual
Compose, Caddyfile y .env cifrado	Diaria y en cada cambio	restic / Git privado	R2 y repositorio	Igual
Base de negocio (Supabase)	Diaria	pg_dump con rol de solo lectura	R2 (cifrado)	Igual
Comprobantes e imágenes (Storage)	Diaria	Sincronización por API de Storage	R2 (cifrado)	Igual
Servidor completo	Diaria	Copia del proveedor	Proveedor	Según plan

Nota. Elaboración propia. El método exacto de sincronización de Storage se confirmará en la III Unidad.

Verificación. Cada semana se ejecuta una verificación de integridad de las copias (restic check) y, cada mes, una restauración completa en una VPS limpia, cronometrada y documentada. De esta forma se concilian la prueba mensual definida en el diseño y la verificación frecuente que recomienda el documento de arquitectura.

Presupuesto de tiempo del RTO (estimado): crear o restaurar el servidor, 15 min; aplicar la configuración base y Compose, 20 min; restaurar bases y volúmenes, 30 min; validar servicios y flujos, 20 min; actualizar DNS y pruebas de humo, 15 min. Total estimado: 100 min, dentro del límite de 120 min, que se medirá en PT-20.

Regla 3-2-1. Los datos del negocio existen en tres copias (producción, copia diaria del proveedor del Droplet y copia cifrada de restic), en dos tipos de almacenamiento (disco del proveedor y almacenamiento de objetos) y una de ellas está fuera del servidor y del proveedor de la VPS (Cloudflare R2). Esta organización corresponde al control A.8.13 de ISO/IEC 27001:2022 (ISO/IEC, 2022a).

8.5 Monitoreo y auditoría

La Tabla 21 define qué se vigila, el umbral de alerta y la acción asociada.

Tabla 21
Monitoreo, umbrales y alertas

Qué se vigila	Umbral de alerta	Acción
CPU, memoria y disco del servidor	CPU > 80 % por 10 min; RAM > 85 %; disco > 80 %	Revisar consumo; limpiar; evaluar ampliar el plan o activar el modo cola
Salud de contenedores	Healthcheck fallido o más de 3 reinicios por hora	Revisar registros; aplicar runbook
Disponibilidad externa	Sin respuesta a la sonda de /healthz	Incidente P1
Webhook de Meta	Tasa de errores 5xx > 1 %; latencia p95 > 1 s	Revisar API y base
Cola de eventos (outbox)	Evento pendiente > 5 min o reintentos agotados	Reprocesar; revisar n8n
Mensajes de WhatsApp	Fallos de envío > 5 %	Revisar plantillas y cuenta
Comprobantes	Comprobante sin revisar > 5 min	Avisar al responsable de caja
Certificados TLS	Vencimiento < 14 días	Verificar renovación de Caddy
Respaldos	Sin copia exitosa en 26 h o verificación fallida	Repetir; incidente P2
Seguridad	Intentos de SSH fallidos repetidos; cambios en /etc o claves	Revisar bitácoras; bloquear origen

Nota. Elaboración propia. Las alertas críticas no dependen de n8n: se envían por correo desde el proveedor y la sonda externa.

Auditoría. La API registra en una tabla de auditoría quién cambió cada estado, desde qué cuenta y a qué hora, y qué comprobante respaldó la aprobación del pago. En el servidor, journald, auditd y los registros de Caddy, n8n y la API se rotan y se conservan 90 días; la bitácora de auditoría del negocio se conserva 12 meses. El auditor revisa semanalmente los intentos de acceso fallidos, los cambios de rol y las aprobaciones de pago manuales. En el MVP no se despliega un SIEM (sección 5.5): la correlación se hace con las alertas del proveedor, auditd y la revisión semanal, lo que cubre los controles A.8.15 y A.8.16 de ISO/IEC 27001:2022 en un nivel proporcional al tamaño del servicio.

9. DISEÑO OPERATIVO

9.1 Roles y responsabilidades

La operación se reparte entre cinco roles. En el proyecto académico, los roles técnicos serán asumidos por los integrantes del equipo; en producción corresponden al personal del negocio y al proveedor de soporte. La Tabla 22 usa el esquema RACI: R, responsable de ejecutar; A, quien rinde cuentas; C, consultado; I, informado.

Tabla 22
Matriz de responsabilidades (RACI)

Actividad	Propietario	Admin. de sistemas	Desarrollo	Caja	Cocina
Aprobar cambios y versiones	A	R	C	I	I
Monitorear y operar la VPS	I	A/R	C	—	—
Verificar pagos y comprobantes	A	I	—	R	—
Gestionar catálogo y precios	A	—	C	R	I
Gestionar accesos y secretos	A	R	C	I	I
Respaldos y restauración	I	A/R	C	—	—
Atender incidentes técnicos	I	A/R	R	I	I
Mantener flujos de n8n y la API	I	C	A/R	—	—
Operar el KDS y avisar el estado	I	—	—	C	A/R

Nota. Elaboración propia.

9.2 Procedimientos de operación

La Tabla 23 fija las tareas periódicas de operación, su responsable y su evidencia.

Tabla 23
Rutina operativa

Frecuencia	Tarea	Responsable	Evidencia
Diaria	Revisar el panel de monitoreo, el estado de los contenedores (docker compose ps) y la última copia exitosa; atender alertas.	Admin. de sistemas	Registro diario
Diaria	Revisar la bandeja de comprobantes pendientes y los pedidos con incidencias.	Caja	Panel
Semanal	Ejecutar restic check; revisar registros de seguridad, intentos de acceso y espacio en disco.	Admin. de sistemas	Informe breve
Semanal	Revisar mensajes fallidos, plantillas y eventos reprocesados.	Desarrollo	Informe breve
Mensual	Restauración completa de prueba, actualización de paquetes y revisión de cuentas y claves.	Admin. de sistemas	Acta de prueba
Trimestral	Revisar capacidad (CPU, RAM, disco, costos de mensajes) y rotar secretos.	Admin. y propietario	Informe

Nota. Elaboración propia.

Arranque y parada controlados. El arranque del servicio se hace con `docker compose up -d` desde el directorio versionado; el estado se confirma con `docker compose ps` (todos en estado healthy) y con una llamada a `/healthz`. La parada ordenada se realiza con `docker compose stop` en este orden: caddy, outbox-worker,

n8n, api y postgres. Tras un reinicio del anfitrión, los servicios deben volver solos en menos de cinco minutos (PT-02).

9.3 Procedimientos de mantenimiento

El mantenimiento programado se describe en la Tabla 24.

Tabla 24
Mantenimiento programado

Tipo	Frecuencia	Procedimiento
Parches de seguridad del SO	Automática	unattended-upgrades; reinicio programado fuera del horario de pedidos si el núcleo lo exige.
Actualización de n8n, API e imágenes	Mensual o ante parche crítico	1) Anunciar ventana. 2) Respaldar y verificar. 3) Descargar la nueva etiqueta fija. 4) Probar en un entorno temporal. 5) Aplicar con docker compose up -d. 6) Ejecutar pruebas de humo. 7) Si falla, volver a la etiqueta anterior y restaurar el respaldo.
Limpieza de recursos	Mensual	Eliminar imágenes y volúmenes huérfanos; revisar poda de ejecuciones y rotación de registros.
Rotación de secretos	90 días	Generar nuevos tokens y secretos compartidos; actualizar .env y reiniciar servicios; registrar el cambio.
Prueba de recuperación	Mensual	Restaurar en una VPS limpia, medir el tiempo y comparar con RTO ≤ 2 h.
Revisión de cuentas y VPN	Mensual	Eliminar usuarios y claves inactivos; verificar roles.

Nota. Elaboración propia.

9.4 Gestión de incidentes

Los incidentes se clasifican por severidad y siguen un ciclo de detección, clasificación, contención, resolución, recuperación y revisión posterior. El ciclo toma como guía las recomendaciones de respuesta a incidentes de NIST (Nelson et al., 2025), alineadas con las funciones Detectar, Responder y Recuperar de NIST CSF 2.0. Los tiempos son objetivos de diseño que se ajustarán según la capacidad del equipo.

La Tabla 25 clasifica los incidentes por severidad y fija los tiempos objetivo; la Tabla 26 resume las acciones inmediatas de los casos más probables.

Tabla 25
Severidad de incidentes y objetivos de atención

Nivel	Ejemplo	Respuesta inicial	Resolución objetivo
P1 Crítica	Servicio caído, pérdida de datos o compromiso de seguridad	15 minutos	≤ 2 horas (RTO)
P2 Alta	Fallo de envío de mensajes, respaldo fallido, disco al 90 %	1 hora	≤ 4 horas
P3 Media	Degradación de rendimiento o error no bloqueante	4 horas	≤ 1 día hábil
P4 Baja	Mejora o consulta	1 día hábil	Planificada

Nota. Elaboración propia.

Para los incidentes más probables se prepararon guías de respuesta iniciales, resumidas en la Tabla 26.

Tabla 26
Guías de respuesta (runbooks) iniciales

Incidente	Síntomas	Acciones inmediatas
VPS inaccesible	La sonda externa falla; no responde la VPN	Revisar el panel del proveedor; reiniciar desde la consola; si persiste, restaurar desde la copia del servidor.
n8n caído	Eventos acumulados en el outbox	Los pedidos no se pierden; reiniciar el contenedor, revisar memoria y registros; el worker reintenta.
Fallo de mensajería	Mensajes sin entregar o rechazados	Revisar estado de Meta, token y plantillas; avisar a caja para contacto manual.
Disco lleno	Alerta de disco > 80 %	Limpiar imágenes, registros y área temporal; revisar poda; ampliar si procede.
Sospecha de acceso indebido	Accesos fallidos repetidos o cambios inesperados	Bloquear el origen, rotar secretos, revocar sesiones y claves, preservar registros y revisar la auditoría.
Comprobante dudoso	Importe o datos no coinciden	Mantener UNDER_REVIEW; contactar al cliente; no confirmar el pedido hasta verificar el movimiento.

Nota. Elaboración propia. Cada incidente P1 y P2 concluye con una breve revisión posterior con causa, acciones y responsables.

10. PLAN DE IMPLEMENTACIÓN

El plan describe cómo se implementará el diseño en la III Unidad. Se organiza en ocho fases para ocho semanas relativas al inicio de la unidad; el calendario real se ajustará a las fechas académicas. Toda la implementación se limita a los componentes ya definidos en este informe; cualquier componente nuevo requerirá una justificación técnica documentada, como exige la II Unidad.

10.1 Actividades y dependencias

La Tabla 27 desglosa el plan de implementación en 19 actividades con su entregable, dependencia y semana.

Tabla 27
Actividades, entregables y dependencias

ID	Actividad	Entregable	Depende de	Semana
A01	Crear cuentas (DigitalOcean, Supabase, Cloudflare, Vercel), dominio y número de WhatsApp de prueba; medir la línea base manual (tiempo de respuesta y pedidos por día)	Cuentas, accesos y línea base	—	S1
A02	Crear el repositorio con la estructura de configuración y la bóveda de secretos	Repositorio base	—	S1
A03	Crear el Droplet Ubuntu 24.04, usuarios y SSH por clave	Servidor accesible	A01	S1–S2

ID	Actividad	Entregable	Depende de	Semana
A04	Endurecer el SO: UFW, fail2ban, parches, auditd, chrony y swap	Línea base de seguridad	A03	S2
A05	Configurar WireGuard, restringir SSH a wg0 y activar el Cloud Firewall	VPN y perímetro	A04	S2
A06	Instalar Docker y Compose; definir redes y límites de recursos	Plataforma de contenedores	A04	S2–S3
A07	Desplegar Caddy con TLS y registros DNS	HTTPS operativo	A01, A06	S3
A08	Desplegar PostgreSQL de n8n y los volúmenes persistentes	Base de n8n	A06	S3
A09	Desplegar la API, conectar con Supabase y habilitar el webhook con firma	API en producción	A07	S3–S4
A10	Implementar la tabla outbox, el worker y los endpoints /internal/*	Eventos durables	A09	S4
A11	Instalar n8n con la configuración de seguridad y el acceso solo por VPN	n8n operativo	A05, A08	S4
A12	Construir y probar los flujos WF-01 a WF-08 con el número de prueba	Flujos en n8n	A10, A11	S4–S5
A13	Panel: bandeja de comprobantes, roles y auditoría	Panel con RBAC	A09	S5
A14	Configurar restic, R2, temporizadores y verificación semanal	Respaldos automáticos	A08	S5
A15	Configurar monitoreo, alertas y sonda externa	Alertas activas	A06	S5–S6
A16	Redactar runbooks y manual de operación	Documentación operativa	A14, A15	S6
A17	Ejecutar el plan de pruebas PT-01 a PT-24 y registrar evidencias	Informe de pruebas	A12–A15	S6–S7
A18	Corregir hallazgos y repetir pruebas fallidas	Sistema validado	A17	S7
A19	Preparar la sustentación, la presentación y el informe final	Entregables finales	A18	S7–S8

Nota. Elaboración propia. Los pesos relativos no representan horas de trabajo.

10.2 Recursos necesarios

Los recursos necesarios se resumen en la Tabla 28.

Tabla 28
Recursos del proyecto

Tipo	Recurso	Observación
Humanos	Cinco integrantes con roles de administración de sistemas, seguridad y redes, desarrollo de API, flujos de n8n y pruebas y documentación.	Dedicación por definir por el equipo
Infraestructura	Droplet de 2 vCPU/4 GB, copia diaria, Supabase, Vercel, Cloudflare R2 y dominio.	S/ 264,19 al mes (sin dominio)

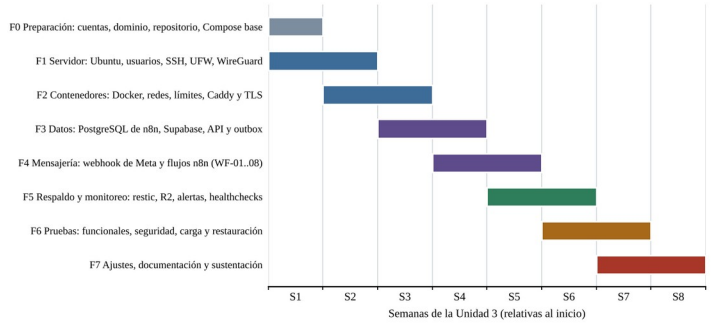
Tipo	Recurso	Observación
Software	Ubuntu, Docker, Caddy, n8n Community, PostgreSQL, WireGuard, restic y herramientas de prueba (nmap, curl, k6 o similar, stress-ng).	Sin costo de licencia
Externos	Cuenta de WhatsApp Business y número de prueba; verificación en Meta.	Posible demora de aprobación
Red local	Router/cortafuegos con VLAN y switch administrable (opcionales).	No incluidos en el presupuesto base

Nota. Elaboración propia.

10.3 Cronograma

La Figura 9 muestra el cronograma propuesto. Las fases F1 a F5 se solapan parcialmente porque los componentes de la capa superior dependen de la plataforma, pero pueden iniciarse en cuanto el componente previo esté estable. Los hitos de control son: servidor endurecido y VPN (fin de S2), HTTPS y API en producción (fin de S4), flujos completos (fin de S5), respaldos y monitoreo (fin de S6) y pruebas aprobadas (fin de S7).

Figura 9
Cronograma de implementación por fases



Nota. Elaboración propia. S1 a S8 son semanas relativas al inicio de la III Unidad.

Dependencias críticas. (1) La VPN debe funcionar antes de cerrar el acceso público por SSH, para no perder el acceso al servidor. (2) El dominio y los registros DNS deben existir antes de emitir los certificados TLS. (3) La verificación de la cuenta de WhatsApp Business depende de Meta y puede retrasar las pruebas con mensajes reales; por eso se trabaja con un número de prueba. (4) El respaldo verificado debe existir antes de la primera actualización de versiones.

11. PLAN DE PRUEBAS

11.1 Estrategia y entorno de pruebas

Las pruebas se ejecutarán en la III Unidad sobre la VPS definitiva, con un número de WhatsApp de prueba y un dominio real, de modo que se valide el diseño en condiciones equivalentes a las de operación. Se organizan en cinco grupos: plataforma y aislamiento (PT-01 a PT-07), seguridad (PT-08 a PT-15), rendimiento y flujo (PT-16 a PT-18), recuperación y mantenimiento (PT-19 a PT-21) y usabilidad y operación (PT-22 a PT-24). Cada prueba se documenta con el comando o procedimiento usado, la evidencia (captura o registro), la fecha, la persona que la ejecutó y el resultado. Las herramientas de prueba (nmap, curl, openssl, stress-ng, una herramienta de carga HTTP y un escáner de secretos) se ejecutan desde un equipo externo o temporal y no forman parte del sistema en producción.

11.2 Casos de prueba

La Tabla 29 reúne los 24 casos de prueba con su procedimiento, resultado esperado y criterio de aceptación.

Tabla 29

Plan de pruebas: qué, cómo, resultado esperado y criterio de aceptación

ID	Qué se prueba	Cómo se prueba	Resultado esperado	Criterio de aceptación
PT-01	Línea base del sistema	lsb_release -a, docker version y docker compose version; compararlas con las versiones de referencia del diseño.	Ubuntu 24.04 LTS y versiones fijadas.	Coinciden con el diseño.
PT-02	Arranque tras reinicio	Reiniciar el anfitrión y medir el tiempo hasta que todos los contenedores estén healthy.	Servicios activos sin intervención.	≤ 5 min.
PT-03	Aislamiento por namespaces	lsns y docker inspect (Privileged, User, CapDrop); ps dentro del contenedor.	Namespaces propios; sin privilegios; no ve procesos del anfitrión.	100 % de contenedores sin privilegios y no root.
PT-04	Límites de cgroups	Generar carga de memoria y CPU en n8n con stress-ng; observar docker stats.	El contenedor se limita o se reinicia por OOM; los demás siguen activos.	Sin caída de otros servicios; webhook < 1 s.
PT-05	Segmentación de redes Docker	Desde caddy conectar a postgres:5432; desde postgres intentar salir a Internet.	Conexión rechazada y sin ruta.	0 conexiones indebidas.
PT-06	Autorrecuperación	docker kill de n8n y de api.	Reinicio automático.	Healthy en ≤ 60 s.
PT-07	Persistencia de datos	docker compose down y up; revisar flujos y base de n8n.	Datos intactos.	Sin pérdida.
PT-08	Exposición de puertos	nmap -sS -sU contra la IP pública desde Internet.	Solo 80/tcp, 443/tcp y 51820/udp.	Ningún otro puerto abierto.
PT-09	SSH y VPN	SSH desde Internet; por wg0 con clave y con contraseña.	Rechazo desde Internet; solo clave por VPN.	100 % de intentos indebidos rechazados.
PT-10	TLS y redirección	openssl s_client o testssl.sh; petición por HTTP.	Certificado válido, TLS ≥ 1.2, redirección a HTTPS.	Sin hallazgos críticos.
PT-11	Verificación del webhook	GET con token correcto e incorrecto; POST con firma válida, inválida y ausente.	200 solo con token y firma válidos; 401/403 en el resto.	100 % de casos correctos.
PT-12	Idempotencia	Enviar dos veces el mismo wamid.	Un solo registro y un solo pedido.	0 duplicados.

ID	Qué se prueba	Cómo se prueba	Resultado esperado	Criterio de aceptación
PT-13	Editor de n8n	Acceder al puerto 5678 desde Internet y desde la VPN.	Inaccesible desde Internet; accesible por VPN con credenciales.	0 accesos públicos.
PT-14	Gestión de secretos	Escáner de secretos sobre el repositorio; ls -l del .env; verificar el respaldo de la llave.	Sin secretos en Git; permisos 600.	0 secretos expuestos.
PT-15	Autorización y regla de pago	Usuario COCINA pide comprobantes; subir una imagen de comprobante; revisar la bitácora de auditoría.	403 para COCINA; pedido en UNDER_REVIEW.	Ningún pedido confirmado sin acción humana.
PT-16	Latencia del webhook	200 solicitudes firmadas con una herramienta de carga HTTP.	Respuesta rápida tras escritura durable.	p95 < 1 s.
PT-17	Carga y capacidad	30 conversaciones simultáneas durante 30 min; vigilar CPU, RAM y errores.	Sistema estable sin pérdida de eventos.	CPU < 70 %, RAM < 80 %, errores < 1 %.
PT-18	Flujo extremo a extremo	50 pedidos de prueba con cocina simulada, incluido un caso de derivación a una persona; medir tiempos y entregas y comparar con la línea base manual.	Pedidos confirmados y notificados.	Confirmación ≤ 2 min; ≥ 95 % de mensajes; ≥ 90 % a tiempo (preparación de 15 a 20 min; entrega según zona y horario).
PT-19	Restauración	Restaurar con restic en una VPS limpia, levantar Compose y validar flujos y credenciales.	n8n opera con sus flujos y credenciales.	Funcional con la llave respaldada.
PT-20	RPO y RTO	Cronometrar PT-19 y comparar la fecha del último dato recuperado.	Servicio recuperado y pérdida acotada.	RTO ≤ 2 h; RPO ≤ 24 h.
PT-21	Actualización y reversión	Actualizar n8n a otra etiqueta y volver a la anterior.	Servicio operativo en ambos sentidos.	Sin pérdida; ≤ 30 min cada sentido.
PT-22	Usabilidad	Cinco operadores usan el panel y el KDS; aplicar CSAT y SUS.	Panel comprensible y rápido.	CSAT ≥ 4/5; SUS ≥ 68.
PT-23	Resiliencia del outbox	Detener n8n tras registrar pedidos y reiniciarlo.	Eventos entregados al volver, sin duplicar efectos.	100 % entregados; 0 duplicados.
PT-24	Alertas	Simular disco > 80 %, contenedor caído y respaldo fallido.	Alertas recibidas.	≤ 5 min desde el evento.

Nota. Elaboración propia a partir de las metas de diseño, de los requerimientos de la sección 4 y de los casos T01 a T20 del documento de arquitectura.

11.3 Criterios de aceptación globales

El diseño se considerará validado cuando: (a) las 24 pruebas se hayan ejecutado y documentado; (b) todas las pruebas relacionadas con requerimientos de prioridad M cumplan su criterio; (c) no existan hallazgos críticos de seguridad abiertos; y (d) las metas de diseño (disponibilidad, tiempo y satisfacción) se hayan medido y comparado con su umbral. Una prueba fallida genera una acción correctiva y se repite. La Tabla 30 vincula cada meta del diseño con su prueba; las métricas de calidad siguen las categorías de ISO/IEC 25010 (ISO/IEC, 2011) y el seguimiento de metas se alinea con los dominios de desempeño del PMBOK 7 (Project Management Institute, 2021).

Tabla 30
Trazabilidad entre metas del diseño y pruebas

Meta	Prueba	Umbral
Disponibilidad 24/7 (reinicio y healthchecks)	PT-02, PT-06, PT-24	Recuperación automática ≤ 5 min y ≤ 60 s
Webhook de Meta < 1 s	PT-16	p95 < 1 s
Eventos durables sin pérdidas	PT-12, PT-23	0 pérdidas, 0 duplicados
Aislamiento por contenedor	PT-03, PT-04, PT-05	Sin privilegios; límites efectivos
Seguridad web 80 y 443 con TLS y cortafuegos	PT-08, PT-10, PT-13	Solo puertos permitidos; TLS válido
RPO ≤ 24 h y RTO ≤ 2 h	PT-19, PT-20	Cumple ambos
Confirmación ≤ 2 min; mensajes ≥ 95 %; a tiempo ≥ 90 % (preparación de 15 a 20 min)	PT-18	Cumple los tres
Satisfacción ≥ 4/5	PT-22	CSAT ≥ 4/5

Nota. Elaboración propia a partir de los requerimientos de la sección 4.

11.4 Verificación analítica previa del diseño

Antes de implementar, se comprobó con un programa en Python (módulo ipaddress y operaciones aritméticas sobre las especificaciones de este informe) que los datos del diseño son coherentes entre sí. La Tabla 31 resume las comprobaciones y sus resultados. Se trata de una verificación de consistencia interna: no mide el comportamiento de un sistema en ejecución ni sustituye a las pruebas PT-01 a PT-24, que se realizarán en la III Unidad.

Tabla 31
Verificación analítica del diseño

N.º	Comprobación	Resultado	Estado
V1	Direccionamiento IP: 9 redes privadas sin solapamientos y dentro de RFC 1918.	0 solapes entre pares; todas las direcciones asignadas pertenecen a su subred y no se repiten; ninguna red coincide con 172.17.0.0/16; 203.0.113.10 pertenece al bloque RFC 5737.	Cumple
V2	Capacidad de hosts por red.	Cada red /24 ofrece 254 direcciones; el mayor rango de DHCP (VLAN 30) usa 201.	Cumple
V3	Memoria: suma de límites de los 5 contenedores.	2 784 MB; con ≈ 700 MB del sistema son 3 484 MB de 4 096 MB (margen de 612 MB) y 2 GB de swap.	Cumple con margen ajustado

N.º	Comprobación	Resultado	Estado
V4	CPU: suma de límites frente a las 2 vCPU.	4,5 vCPU de límite sobre 2 vCPU: los límites son techos y no reservas; la sobreasignación de 2,25 veces se vigila en PT-04 y PT-17.	Cumple con vigilancia
V5	Disco: presupuesto de 80 GB.	40 GB asignados y 40 GB libres (50 %).	Cumple
V6	Costos: recálculo con S/ 3,46702 por US$ 1.	Mensual S/ 264,19; anual S/ 3 170,24 (12 × 264,19 = 3 170,28; diferencia de redondeo S/ 0,04); costo total de propiedad a 36 meses S/ 9 510,84.	Cumple
V7	RTO: suma del presupuesto de tiempo.	15 + 20 + 30 + 20 + 15 = 100 min, frente al límite de 120 min.	Cumple
V8	Exposición: puertos públicos frente a reglas.	Solo 80/tcp, 443/tcp y 51820/udp en la nube y en UFW; el editor de n8n se enlaza a 10.99.0.1, dentro de 10.99.0.0/24.	Cumple
V9	Trazabilidad requisito → prueba.	Los 12 RNF y 18 de los 19 RF (RF-19 queda fuera del MVP) tienen al menos una prueba; las 24 pruebas se asocian a algún requisito.	Cumple
V10	Alternativas de despliegue (sección 5.5).	La alternativa A es primera con los pesos del equipo (4,00) y con pesos iguales (3,83), y en 92,8 % de 10 000 combinaciones aleatorias de pesos.	Cumple

Nota. Elaboración propia. Programa verify.py ejecutado sobre las cifras de las secciones 5 a 12. “Cumple” indica coherencia con el diseño y no desempeño medido.

Quedan fuera de esta verificación analítica el comportamiento de la lógica de negocio, la sintaxis de los archivos de configuración reales y el comportamiento bajo carga, que se comprobaron con el prototipo de la sección 11.5. Tampoco hubo validación con expertos externos, lo que se reconoce como limitación (sección 13).

11.5 Prototipo de verificación del diseño (prueba de concepto)

Además de la verificación analítica, el equipo construyó un prototipo mínimo que traduce a código las decisiones críticas del diseño y las somete a las pruebas de la sección 11.2, para aportar evidencia de validación sin adelantar la implementación de la III Unidad. Usa Node.js 22 con sus módulos estándar, un almacén en memoria con transacciones para la API, PostgreSQL 16 real para el esquema de datos y Docker Compose para validar el despliegue. Incluye la API de negocio con la máquina de estados del pedido, el verificador de firmas del webhook, el *outbox* con reintentos, un simulador del orquestador y dos workflows de n8n exportados (WF-01, enrutador de mensajes, y WF-05, evidencia de pago). El código y la evidencia están en el repositorio del equipo (véase el Resumen) y en el archivo comprimido que acompaña este informe. La Tabla 35, en el Anexo B, detalla los resultados: las 13 pruebas automatizadas pasaron, las diez pruebas de restricciones del esquema rechazaron los datos inválidos y las 17 comprobaciones del Compose coincidieron con las cifras de este informe.

Alcance y límites. El prototipo valida la capa de reglas de negocio, el esquema de datos y la coherencia de la configuración con el diseño; no valida el sistema desplegado. No se levantaron los contenedores, no se ejecutaron los workflows en un n8n real y no se probaron Caddy con certificados reales, WireGuard, la API de WhatsApp Cloud ni la descarga de medios. Las latencias provienen de un entorno de desarrollo y no de la VPS, y el

esquema se probó en PostgreSQL 16 mientras que el Compose fija la versión 17. Las demás pruebas de la sección 11.2 y la medición definitiva de PT-16 y PT-18 quedan para la III Unidad.

12. ANÁLISIS DE FACTIBILIDAD

12.1 Factibilidad técnica

La solución es técnicamente factible por cinco razones. Primero, todos los componentes son maduros y tienen documentación pública. Segundo, el patrón de despliegue (n8n y proxy inverso en contenedores sobre una VPS Linux) está documentado en la literatura revisada (Muñoz Ingaruca, 2026; Chen et al., 2026; Casaclang et al., 2026; Patiño et al., 2026). Tercero, el dimensionamiento coincide con el de un diseño publicado (2 vCPU, 4 GB y 80 GB), y la carga esperada es baja (2 000 pedidos al mes son unos 270 mensajes por día). Cuarto, el crecimiento tiene una ruta definida: escalado vertical o modo cola con Redis y workers (Venkiteela, 2025). Quinto, la dependencia de la plataforma de Meta se reduce usando el canal oficial y aislando la lógica de negocio en la API.

La Tabla 32 evalúa la factibilidad técnica por criterio.

Tabla 32
Evaluación técnica por criterio

Criterio	Evaluación	Riesgo o condición
Madurez del software	Componentes con soporte de largo plazo y comunidad activa.	Fijar versiones y verificar vigencia.
Capacidad del servidor	Suma de límites de memoria de 2 784 MB más ≈ 0,7 GB del sistema, con 4 GB y swap.	Margen ajustado si n8n supera 1 GB; medir en PT-17.
Competencias del equipo	Contenidos de SO II: procesos, contenedores, redes, seguridad y respaldos.	Curva de aprendizaje de WireGuard, restic y Compose.
Dependencias externas	Meta, Supabase, Vercel, Cloudflare y DigitalOcean.	Cambios de tarifas y límites; respaldo propio de datos.
Latencia	Webhook < 1 s depende de la región del servidor.	Medir latencia hacia Tacna antes de fijar la región.
Seguridad y recuperación	Controles por capas y restauración documentada.	Validar en PT-08 a PT-15 y PT-19 a PT-20.

Nota. Elaboración propia.

12.2 Factibilidad económica

El costo recurrente de infraestructura es de **S/ 264,19 mensuales** (S/ 3 170,24 anuales, según los precios de referencia del equipo). La licencia de software es S/ 0, aunque n8n Community se distribuye bajo la Sustainable Use License (código disponible, no software libre en sentido estricto), por lo que debe revisarse si el negocio se ofrece a terceros como servicio (Flores Chino et al., 2026).

La Tabla 33 calcula el presupuesto de infraestructura en soles.

Tabla 33
Presupuesto mensual y anual de infraestructura

Concepto	US$ ref.	S/ mensual	S/ anual
DigitalOcean VPS Basic (2 vCPU, 4 GB, 80 GB)	24,00	83,21	998,50

Concepto	US$ ref.	S/ mensual	S/ anual
Copia diaria del servidor (30 % del Droplet)	7,20	24,96	299,55
Vercel, plan comercial (tienda y panel)	20,00	69,34	832,08
Supabase (pedidos y archivos)	25,00	86,68	1 040,11
Cloudflare R2 (hasta 10 GB)	0,00	0,00	0,00
Licencias de software	0,00	0,00	0,00
Total	—	264,19	3 170,24

Nota. Elaboración propia con precios referenciales del equipo; tipo de cambio referencial S/ 3,46702 por US$ 1 (noviembre de 2026). Los importes anuales se calculan sobre los dólares sin redondear (US$ × 12 × 3,46702) y su suma es S/ 3 170,24; 12 veces el total mensual redondeado da S/ 3 170,28, y la diferencia de S/ 0,04 es un efecto del redondeo. No incluye dominio, impuestos, soporte, mensajes de WhatsApp ni equipos de red opcionales.

El costo variable principal es la mensajería. Con el modelo del documento de arquitectura (Equipo RETEQUE-OS, 2026), que supone cuatro mensajes de servicio por pedido, 1 000 mensajes gratuitos al mes y US$ 0,03 por mensaje adicional, el costo total depende del volumen (Tabla 34). Las tarifas de Meta deben confirmarse en su lista oficial vigente antes de contratar.

Tabla 34
Escenarios de costo mensual con mensajes de WhatsApp

Pedidos/ mes	Mensajes	Facturables	US$ WhatsApp	S/ WhatsApp	S/ total	S/ por pedido
100	400	0	0	0,00	264,19	2,64
500	2 000	1 000	30	104,01	368,20	0,74
2 000	8 000	7 000	210	728,07	992,26	0,50

Nota. Cálculo propio: mensajes = 4 × pedidos; facturables = mensajes − 1 000 (mínimo 0); US$ = 0,03 × facturables; conversión a S/ 3,46702. Excluye IA, dominio e impuestos.

El costo medio por pedido baja de S/ 2,64 a S/ 0,50 al crecer el volumen, porque la infraestructura es un costo fijo. No se dispone de datos de ingresos ni de márgenes del negocio, por lo que el beneficio no se cuantifica en este informe. El ahorro de tiempo y el efecto sobre la satisfacción se medirán en la III Unidad; los estudios previos (Muñoz Ingaruca, 2026; Patiño et al., 2026) no se extrapolan a este caso.

Costo total de propiedad (TCO). Con el horizonte de 36 meses que usan Chen et al. (2026), el gasto en infraestructura es de S/ 9 510,84 (36 × 264,19) sin mensajes de WhatsApp, S/ 13 255,20 con 500 pedidos mensuales y S/ 35 721,36 con 2 000. El gasto de capital (CAPEX) del presupuesto base es S/ 0, porque no se compra hardware; los equipos de red opcionales no están cotizados. El VAN, la TIR y la relación beneficio-costo no se calculan por falta de datos de ingresos.

Costos no incluidos: dominio, impuestos, horas del equipo, soporte, el router y el switch para las VLAN, y la IA (menos de US$ 1 al mes en el escenario de 2 000 pedidos, fuera del MVP).

12.3 Factibilidad operativa

La operación es viable con un equipo pequeño porque las tareas rutinarias son pocas (Tabla 23), la mayoría está automatizada (parches, respaldos, renovación de certificados y reinicios) y las alertas se concentran en un punto. Los operadores solo usan el panel, el KDS y WhatsApp, lo que limita la capacitación a los comprobantes, los estados y la derivación a una persona. Los factores que condicionan la factibilidad operativa son: (1) la disponibilidad

de una persona que verifique los comprobantes en pocos minutos, dado que la regla de pago exige verificación humana; (2) la continuidad del conocimiento técnico, para lo cual se documentan runbooks y diagramas; y (3) la aceptación de los clientes, que se medirá con la encuesta de satisfacción.

13. CONCLUSIONES

Las conclusiones responden a cada objetivo específico de la sección 3.2.

1. **Requerimientos.** Se especificaron 19 requerimientos funcionales y 12 no funcionales con prioridad, métrica y prueba asociada; las metas de disponibilidad, webhook, RPO y RTO permiten verificar el diseño en la III Unidad, y la verificación analítica confirmó que todas las pruebas se asocian a algún requisito.
2. **Arquitectura.** La solución integra la tienda web, el panel, la API de negocio, n8n y WhatsApp bajo el principio de que n8n coordina y la API decide, de modo que las reglas de negocio, los precios y la verificación de pagos quedan fuera del orquestador. Cada decisión se contrastó con alternativas descartadas (Tabla 5) y con la evidencia de la I Unidad (Tabla 3); diez figuras la documentan, con los cuatro diagramas exigidos.
3. **Infraestructura.** El aislamiento usa *namespaces* y *cgroups* de Docker con contenedores sin privilegios; los límites de memoria suman 2 784 MB sobre 4 GB, margen ajustado que se medirá en PT-17.
4. **Red.** Nueve segmentos sin solapamientos combinan la VPC, tres redes Docker y cuatro VLAN; a Internet solo se exponen los puertos 80, 443 y 51820/udp, y la administración se hace por VPN WireGuard y SSH con claves.
5. **Seguridad y recuperación.** Los 12 riesgos tienen controles asociados al Anexo A de ISO/IEC 27001:2022, una prueba que los verifica y una alineación con NIST CSF 2.0. El respaldo cifrado diario hacia Cloudflare R2 (regla 3-2-1) sustenta RPO ≤ 24 h y RTO ≤ 2 h.
6. **Operación, implementación y factibilidad.** El diseño operativo, el plan de ocho semanas con 19 actividades y los 24 casos de prueba traducen el diseño en tareas y criterios de aceptación. La solución es factible: la infraestructura cuesta S/ 264,19 al mes y los mensajes de WhatsApp añaden entre S/ 0 y S/ 728 según el volumen.

Limitaciones. Este informe es un diseño con evidencia propia solo parcial: la verificación es analítica y de prototipo; los puntajes de la matriz son un juicio del equipo; los precios y versiones deben confirmarse al contratar; no hay línea base local ni datos de ingresos ni validación con expertos externos. El prototipo (sección 11.5) cubre la lógica de negocio, el esquema de datos y la configuración, no el sistema desplegado. **Trabajo futuro.** La III Unidad implementará y medirá lo diseñado en el entorno real (PT-01 a PT-24), levantará la línea base manual y evaluará el modo cola con Redis, la IA acotada y el equipo de red para las VLAN.

14. REFERENCIAS BIBLIOGRÁFICAS

Alvarado Huamán, B. J. J., Saenz Cabanillas, A. G., y Sierra-Liñan, F. (2025). AI-powered chatbot integrated with WhatsApp for automatic invoice generation in a service company. *5th LACCEI International Multiconference on Entrepreneurship, Innovation and Regional Development (LEIRD 2025)*. LACCEI. https://doi.org/10.18687/LEIRD2025.1.1.802

Amir, A. R., y Atif, S. M. (2026). *Evaluating workflow automation efficiency using n8n: A small-scale business case study* [Prepublicación]. arXiv. https://arxiv.org/abs/2602.01311

Andaluz Freire, J. A., y Ulloa Gavilanes, C. M. (2026). *Experiencia de usuario en la automatización de pedidos mediante la aplicación n8n* [Trabajo de titulación, Universidad Técnica de Ambato]. https://repositorio.uta.edu.ec/items/86c5a596-de08-4a59-a697-4e8f34e4006d

Andrade Andrade, O. S. (2025). *Desarrollo de un asistente inteligente para la atención automatizada de clientes en la Cooperativa de Ahorro y Crédito Yuyay Ltda.* [Trabajo de titulación, Universidad Católica de Cuenca]. https://dspace.ucacue.edu.ec/items/29643e7c-a488-4a39-a893-c6de48dff5f9

Arkko, J., Cotton, M., y Vegoda, L. (2010). *IPv4 address blocks reserved for documentation* (RFC 5737). Internet Engineering Task Force. https://doi.org/10.17487/RFC5737

Bui, T. (2015). *Analysis of Docker security* [Trabajo de seminario, Aalto University]. arXiv. https://arxiv.org/abs/1501.02967

Casaclang, H., Ionescu, B., Kim, Y., y Jo, J.-Y. (2026). Self-hosted workflow automation for AI-based cybersecurity operation. *Journal of The Colloquium for Information Systems Security Education, 13*(1). https://journal.cisse.info/index.php/jcisse/article/download/241/241

Chen, D.-F., Chen, Y.-H., y Chen, B.-S. (2026). Designing a continuous operational feedback loop for direct-to-consumer commerce: Integrating event-driven automation and on-premise generative AI. *Information, 17*(7), 628. https://doi.org/10.3390/info17070628

de Freitas, B. A. T., y Lotufo, R. de A. (2024). *Retail-GPT: Leveraging retrieval augmented generation (RAG) for building e-commerce chat assistants* [Prepublicación]. arXiv. https://arxiv.org/abs/2408.08925

Donenfeld, J. A. (2017). WireGuard: Next generation kernel network tunnel. *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*. Internet Society. https://doi.org/10.14722/ndss.2017.23160

Equipo RETEQUE-OS. (2026). *Arquitectura de automatización de pedidos por WhatsApp con n8n y VPS* (Versión 1.0) [Documento de diseño técnico, 26 de octubre]. Universidad Privada de Tacna.

Fendley, N., Liu, Z., Guan, A., Zhong, J., y Cao, Y. (2026). *Comment and control: Hijacking agentic workflows via context-grounded evolution* [Prepublicación]. arXiv. https://arxiv.org/abs/2605.11229

Fitriani, I., y Suyatno, D. F. (2026). Development of an AI-based customer service information system for MSME XYZ. *Journal of Emerging Information Systems and Business Intelligence, 7*(3), 478–491. https://doi.org/10.26740/jeisbi.v7i3.78204

Flores Chino, M. H., Dongo Palza, M. A., Tarqui Collatupa, Y. M., Hinojosa CalloApaza, K. E., y Halanocca Rojas, U. D. (2026). *Automatización inteligente de procesos mediante n8n: Arquitectura, evolución e integración con WhatsApp, Telegram y aplicaciones web* [Artículo de la I Unidad, Sistemas Operativos II]. Universidad Privada de Tacna.

Fuady, M. D., y Tundo. (2025). Optimalisasi teknoologi n8n dalam pengembangan aplikasi penilaian CV ATS-compliant untuk evaluasi kelayakan siswa SMK. *Jurnal Tekno Kompak, 19*(2), 142–154. https://doi.org/10.33365/jtk.v19i2.72

Ghosh, S., y Mittal, G. (2025). *Low code RAG-LLM framework for context-aware querying in electrical standards, design, and research* [Prepublicación no revisada por pares]. Preprints.org. https://doi.org/10.20944/preprints202507.0537.v1

Gomez-Maguiña, A., y Sierra-Liñan, F. (2024). Análisis de la inteligencia artificial en la automatización de procesos comerciales de las empresas: Una revisión sistemática de la literatura científica. *4th LACCEI International Multiconference on Entrepreneurship, Innovation and Regional Development (LEIRD 2024)*. LACCEI. https://doi.org/10.18687/LEIRD2024.1.1.288

Hudeček, V., y Dušek, O. (2023). *Are large language models all you need for task-oriented dialogue?* [Prepublicación]. arXiv. https://arxiv.org/abs/2304.06556

Institute of Electrical and Electronics Engineers. (2022). *IEEE standard for local and metropolitan area networks—Bridges and bridged networks* (IEEE Std 802.1Q-2022). https://doi.org/10.1109/IEEESTD.2022.10004498

ISO/IEC. (2011). *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

ISO/IEC. (2022a). *ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. International Organization for Standardization.

ISO/IEC. (2022b). *ISO/IEC 27005:2022 Information security, cybersecurity and privacy protection — Guidance on managing information security risks*. International Organization for Standardization.

Jondec Delgado, J., Zavaleta Galarza, E., Venturo Ramos, A., y Mendoza de los Santos, A. C. (2026). Agente telefónico automatizado para gestión de reservas en restaurantes: Desarrollo y evaluación de usabilidad. *Revista Innovación y Software, 7*(1), 49–63. https://doi.org/10.48168/innosoft.s29.a296

Moreno, M. S. (2025). Automatización inteligente para procesos correctos: n8n como motor de eficiencia digital. *Revista Innovación y Software, 6*(2), 58–73. https://doi.org/10.48168/innosoft.s24.a321

Moreno Murillo, E., Batista Hernández, N., y Leyva Vázquez, M. (2026). Arquitectura empresarial basada en TOGAF para la optimización de procesos CRM mediante automatización con n8n en la Unidad Educativa Ecomundo Babahoyo. *Magazine de las Ciencias, 11*(1). https://doi.org/10.5281/zenodo.21382453

Muñoz Ingaruca, C. P. (2026). *Diseño e implementación de un sistema automatizado mediante un bot conversacional y APIs para la atención del cliente en una constructora inmobiliaria* [Tesis de licenciatura, Universidad Nacional del Centro

del Perú]. https://repositorio.uncp.edu.pe/server/api/core/bitstreams/6786d141-fb28-49fa-907b-d715fc9bca61/content

National Institute of Standards and Technology. (2024). *The NIST cybersecurity framework (CSF) 2.0* (NIST CSWP 29). https://doi.org/10.6028/NIST.CSWP.29

Nelson, A., Rekhi, S., Souppaya, M., y Scarfone, K. (2025). *Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile* (NIST SP 800-61r3). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-61r3

Nguyen, T. H., Trivedi, R. H., Fukukawa, K., y Adomako, S. (2025). Investigating drivers of customer experience with virtual conversational agents. *Information Systems Frontiers, 27*(5), 1871–1897. https://doi.org/10.1007/s10796-024-10572-0

Nuñez Cartolin, C. A. (2021). *Chatbot en la mejora del proceso de ventas en la empresa Newocean Technology S.A.C., Lima 2021* [Tesis de maestría, Universidad César Vallejo]. https://hdl.handle.net/20.500.12692/71723

Pandey, S., y Lavanya, V. (2025). Automated skill enhancement system using Telegram bot and n8n workflow automation. *International Journal of Creative Research Thoughts, 13*(9), a77–a88. https://ijcrt.org/viewpaperforall.php?paper=IJCRT2509009

Patiño, S., Quiñonez, J., Rojas, J., y Quiñonez, N. (2026). Automatización del proceso de integración académica entre SIAD y Moodle mediante n8n: Estudio de caso en una universidad latinoamericana. *Revista Multidisciplinar de Estudios Generales, 5*(3), 6255–6271. https://doi.org/10.70577/reg.v5i3.950

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK® guide) and the standard for project management* (7.ª ed.). Project Management Institute.

Ramadhani, A., Yantoro, M. D., Akmal, M. F., Mahfud, M., y Fauzi. (2025). Chatbot otomatis dengan n8n dan AI untuk analisis data dan pelaporan hasil. *Jurnal Riset Teknik Komputer, 2*(2), 18–23. https://doi.org/10.69714/x1p94182

Rekhter, Y., Moskowitz, B., Karrenberg, D., de Groot, G. J., y Lear, E. (1996). *Address allocation for private internets* (RFC 1918). Internet Engineering Task Force. https://doi.org/10.17487/RFC1918

Ruhallah, M. L., Pratami, R., y Gozali, A. A. (2025). Utilizing GPT-4o Mini in designing a WhatsApp chatbot to support the new student admission process at Telkom University. *Journal of Dinda, 5*(2), 176–189. https://doi.org/10.20895/dinda.v5i2.1963

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., y Scialom, T. (2023). *Toolformer: Language models can teach themselves to use tools* [Prepublicación]. arXiv. https://arxiv.org/abs/2302.04761

Siregar, R. P., Adam, R. I., y Rozikin, C. (2026). Implementasi n8n workflow untuk optimalisasi chatbot desain berbasis Gemini API. *Jurnal Informatika dan Teknik Elektro Terapan, 14*(2). https://doi.org/10.23960/jitet.v14i2.9374

Sisephaputra, B., Permatasari, R., Alit, R., Suroni, A., Saputra, R. N., Orvala, F., y Ramadhan, D. G. (2026). Event-driven conversational workflow architecture for academic service automation using n8n. *International Journal of Computer, Network Security and Information System, 7*(2), 17–24. https://doi.org/10.33005/ijconsist.v7i2.189

Tang, Y., Zhou, Y., y Chen, H. (2026). *Characterizing large language model agentic workflows: A study on n8n ecosystem* [Prepublicación]. arXiv. https://arxiv.org/abs/2606.29116

Ureta Ureta, J. I. (2025). *Propuesta para implementar IA en el sistema e-commerce en la empresa Escuela de Estrategia, Argumentación e Inteligencia Artificial para mejorar su competitividad* [Trabajo de investigación, Escuela ISIL]. https://hdl.handle.net/20.500.14858/1918

Venkiteela, P. (2025). n8n: An open-source workflow automation platform for enterprise integration and AI-driven orchestration. *International Journal of Computer Applications, 187*(63). https://doi.org/10.5120/ijca2025926031

Vinothraj, P., Sarafin, X. A., Geethapriya, A., Alagumalai, N., y Jain, M. D. (2026). Automation builder: Natural language-driven workflow generation using large language model and n8n. *International Journal of Engineering Research & Technology*, *15*(5). https://www.ijert.org/research/automation-builder-natural-language-driven-workflow-generation-using-large-language-model-and-n8n-IJERTV15IS050297.pdf

Vishwakarma, V. K., Bhargava, R., y Morya, S. (2025). A review paper on AI driven workflow agent in n8n for intelligent task automation. *International Journal of Creative Research Thoughts*, *13*(10), b971–b980. https://ijcrt.org/viewpaperforall.php?paper=IJCRT2510239

Wali, M., Nasir, N., y Iqbal, T. (2025). Implementing workflow automation with n8n to enhance operational efficiency and performance in the Sharia Cooperative of Bank Indonesia, Aceh Province. *Journal Digital Technology Trend*, *4*(1), 36–47. https://doi.org/10.56347/jdtt.v4i1.341

Wang, Y., Tian, Y., Shen, X., Zhang, G., Sun, J., Zhang, H., Xu, R., y Zhao, F. (2025). *Fault2Flow: An AlphaEvolve-optimized human-in-the-loop multi-agent system for fault-to-workflow automation* [Prepublicación]. arXiv. https://arxiv.org/abs/2511.12916

Wati, V., y Nugrahantoro, A. (2026). Development of a WhatsApp-based disaster mitigation chatbot using n8n workflow automation. *ELKOM: Jurnal Elektronika dan Komputer*, *19*(1), 472–482. https://doi.org/10.51903/elkom.v19i1.3835

Widiyanti, N., Hasanuddin, T., y Azis, H. (2026). An LLM-based AI task agent for academic task management with n8n and Telegram. *Indonesian Journal of Data and Science*, *7*(2), 363–375. https://doi.org/10.56705/ijodas.v7i2.412

Wu, Z., Wang, X., Pang, R. Y., Wang, C., y Feit, A. M. (2026). *From review to reuse: How post-task workflow can support human-AI agent interaction* [Prepublicación]. arXiv. https://arxiv.org/abs/2609.13136

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., y Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

Yu, C., Cheng, Z., Cui, H., Gao, Y., Luo, Z., Wang, Y., Zheng, H., y Zhao, Y. (2025). *A survey on agent workflow: Status and future* [Prepublicación]. arXiv. https://arxiv.org/abs/2508.01186

Zaga Palomino, C. A., y Chire Vilcarima, A. K. (2025). Artificial intelligence in process automation for competitive strategy in logistics: RSL. *23rd LACCEI International Multi-Conference for Engineering, Education, and Technology (LACCEI 2025)*. LACCEI. https://doi.org/10.18687/LACCEI2025.1.1.417

ANEXO A. CONFIGURACIONES DE REFERENCIA

Los siguientes extractos ilustran cómo se implementarán los controles del diseño. Los valores entre ángulos (<...>) son marcadores que se sustituirán durante la III Unidad; los comandos se validarán contra la documentación de las versiones fijadas.

A.1 Caddyfile (proxy inverso con TLS)

```
api.<dominio> {
  encode zstd gzip
  header {
    Strict-Transport-Security "max-age=31536000; includeSubDomains"
    X-Content-Type-Options "nosniff"
    -Server
  }
  reverse_proxy api:3000
}
# El editor de n8n no tiene bloque público: se accede por
10.99.0.1:5678 (VPN).
```

A.2 Cortafuegos del anfitrión (UFW y cadena DOCKER-USER)

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 80/tcp
ufw allow 443/tcp
ufw allow 51820/udp
ufw allow in on wg0 to any port 22 proto tcp
ufw enable
# Docker esquiva UFW: se filtra en DOCKER-USER (eth0 = interfaz
pública)
iptables -I DOCKER-USER -i eth0 -j DROP
iptables -I DOCKER-USER -i eth0 -p tcp -m conntrack --ctorigdstport 80
-j ACCEPT
iptables -I DOCKER-USER -i eth0 -p tcp -m conntrack --ctorigdstport 443
-j ACCEPT
iptables -I DOCKER-USER -m conntrack --ctstate ESTABLISHED,RELATED -j
ACCEPT
# Persistir con iptables-persistent o una unidad systemd posterior a
docker.service
```

A.3 VPN WireGuard y SSH

```
# /etc/wireguard/wg0.conf (servidor)
[Interface]
Address = 10.99.0.1/24
ListenPort = 51820
PrivateKey = <clave-privada-servidor>
[Peer]   # administrador 1
PublicKey = <clave-publica-admin-1>
AllowedIPs = 10.99.0.10/32

# /etc/ssh/sshd_config.d/10-retequeos.conf
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
AllowUsers <usuario-admin>
MaxAuthTries 3
ListenAddress 10.99.0.1   # ssh debe arrancar después de wg-quick@wg0
# Ubuntu 24.04 usa ssh.socket: comprobar con 'ss -tlnp | grep :22' que
solo escucha en wg0
```

A.4 Respaldo diario con restic y temporizador systemd

```
# /usr/local/sbin/respaldo.sh (resumen; se ejecuta como root)
set -euo pipefail
docker compose -f /srv/compose/compose.yml exec -T postgres \
       pg_dump -U n8n -Fc n8n > /srv/respaldo/n8n.dump
# /srv/compose incluye .env con la clave de cifrado de n8n (respaldo
imprescindible)
restic backup /srv/respaldo /srv/compose /etc/wireguard
restic forget --keep-daily 7 --keep-weekly 4 --keep-monthly 3 --prune

# /etc/restic.env (permisos 600):
RESTIC_REPOSITORY=s3:https://<cuenta>.r2.cloudflarestorage.com/<bucket>
RESTIC_PASSWORD_FILE=/root/.restic-pass
AWS_ACCESS_KEY_ID=<clave-r2>   AWS_SECRET_ACCESS_KEY=<secreto-r2>

# /etc/systemd/system/respaldo.service
[Service]
Type=oneshot
EnvironmentFile=/etc/restic.env
ExecStart=/usr/local/sbin/respaldo.sh

# /etc/systemd/system/respaldo.timer
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
[Install]
WantedBy=timers.target
```

ANEXO B. EVIDENCIA DEL PROTOTIPO

Este anexo indica dónde está la evidencia que respalda la sección 11.5 y cómo repetirla. La Tabla 35 detalla los resultados obtenidos y la Tabla 36 relaciona cada prueba del diseño con su archivo de evidencia. Los archivos de salida se generan con el script run_all.sh y se guardan en la carpeta evidence del repositorio.

Tabla 35
Resultados del prototipo de verificación del diseño

N.º	Elemento verificado	Resultado observado	Estado
P1	Verificación del webhook de Meta (PT-11): reto GET y firma HMAC-SHA256 sobre el cuerpo sin procesar.	Reto aceptado solo con el token correcto; firma inválida o ausente rechazada.	Cumple
P2	Idempotencia por wamid (PT-12) y creación única de la orden (T05).	El mismo mensaje enviado dos veces genera un solo registro y una sola orden.	Cumple
P3	Regla de pago (PT-15), roles y auditoría.	El comprobante deja el pedido en revisión y el pago en PENDING; solo el cajero confirma; cada cambio queda en la auditoría.	Cumple
P4	Casos T01 a T09 y T12 (cotización, producto inexistente, delivery sin dirección, comprobante repetido o reutilizado, atención humana).	13 de 13 pruebas pasan; el total se calcula en el servidor y no se acepta lo que el catálogo no tiene.	Cumple
P5	Resiliencia del *outbox* (PT-23) y atomicidad.	Con n8n caído los eventos esperan y se entregan con *backoff* al recuperarse; si falla la transacción no quedan filas a medias.	Cumple
P6	Esquema PostgreSQL y restricciones S1 a S10.	10 tablas; unicidad, CHECK, claves foráneas, atomicidad y auditoría de solo inserción rechazan los datos inválidos; respaldo y restauración íntegros en una base pequeña.	Cumple
P7	Compose y red (comprobaciones C1 a C17).	17 de 17: imágenes con versión fija, solo 80 y 443 públicos, n8n enlazado a 10.99.0.1, sin privilegios, 2 784 MB de límites.	Cumple
P8	Workflows de n8n WF-01 y WF-05 (análisis estático).	JSON válido, nodos alcanzables, sin secretos incrustados, reintentos y claves de idempotencia; el acuse nunca dice «pagado».	Cumple (sin ejecutar en n8n)
P9	Ensayos parciales de PT-16 y PT-18.	200 de 200 webhooks con 200 OK (p95 21,6 ms, p99 40,0 ms); 50 pedidos simulados: 50 confirmados, 0 pagos duplicados, 0 eventos sin entregar.	Indicativo

Nota. Elaboración propia a partir de la evidencia del prototipo (Anexo B). Entorno: Node.js 22, 2 CPU, 8 GB de RAM; no es la VPS del diseño.

Tabla 36
Relación entre los casos de prueba del diseño y la evidencia del prototipo

Prueba del diseño	Evidencia del prototipo	Archivo
PT-11, PT-12, PT-15, PT-23	13 pruebas con node --test (firmas, idempotencia, regla de pago, outbox, atomicidad, máquina de estados).	evidence/test_results.txt
PT-18 (parcial)	Flujo de un pedido de la cotización a la confirmación, y ensayo de 50 pedidos simulados.	evidence/e2e_demo.txt; evidence/load_test.txt
PT-16 (parcial)	200 webhooks firmados contra la API local.	evidence/load_test.txt
PT-19 (parcial)	Pruebas S1 a S10, volcado y restauración con pg_dump y pg_restore en PostgreSQL 16.15.	evidence/sql_check.txt
PT-03 a PT-05, PT-08 (análisis del Compose)	Comprobaciones C1 a C17 sobre infra/compose.yaml con docker compose config.	evidence/check_infra.txt
WF-01 y WF-05	Análisis estático de los workflows exportados.	evidence/check_workflows.txt

Nota. Elaboración propia. Los nombres de archivo son rutas dentro del repositorio del prototipo.

Procedimiento para repetir las pruebas (requiere Node.js 22, PostgreSQL 16 y Docker CLI con el plugin Compose):

```
cd restaurante-whatsapp-proto
bash scripts/run_all.sh        # pruebas, flujo, carga, SQL, Compose y
workflows
cat evidence/test_results.txt  # 13 de 13 pruebas superadas
```